

CS-305-MJ-P Operating System-I - Practical Slips Solved

T.Y.B.Sc.(C.S.) NEP-2020, Sem-V, SPPU. All 30 slips: question paper image, complete C program(s), and a tested sample run using the slip's own data. Compile with: `gcc file.c -o out` (Linux). Q3 (viva) answers are on the last pages.

Slip	Q1	Q2
1	Banker's algorithm - menu driven	Shell with 'count' command
2	Linked file allocation	Disk scheduling - FCFS
3	Banker's algorithm - Need matrix & safe sequence	Disk scheduling - SSTF
4	Banker's algorithm - menu driven	Disk scheduling - SCAN
5	Page replacement - FIFO	Resource-Request algorithm
6	Sequential file allocation	Disk scheduling - C-SCAN
7	Resource-Request algorithm	<code>nice()</code> system call with <code>fork()</code>
8	Indexed file allocation	<code>nice()</code> system call with <code>fork()</code>
9	Page replacement - Optimal	CPU scheduling - Non-preemptive SJF
10	CPU scheduling - Round Robin	Disk scheduling - LOOK
11	Banker's algorithm - menu driven	Page replacement - Optimal
12	CPU scheduling - FCFS	<code>fork()</code> : bubble sort (parent) + insertion sort (child)
13	CPU scheduling - Non-preemptive SJF	Disk scheduling - LOOK
14	<code>fork()</code> + <code>execve()</code> : binary search	Page replacement - FIFO
15	Orphan process	CPU scheduling - Preemptive Priority
16	Page replacement - LRU (counter)	Shell with 'count' command
17	CPU scheduling - Non-preemptive SJF	Sequential file allocation
18	CPU scheduling - Preemptive SJF (SRTF)	Orphan process
19	CPU scheduling - Preemptive Priority	<code>nice()</code> system call with <code>fork()</code>
20	Disk scheduling - LOOK	Page replacement - LRU (counter)
21	Page replacement - MFU	<code>nice()</code> system call with <code>fork()</code>
22	Linked file allocation	CPU scheduling - Non-preemptive Priority
23	<code>fork()</code> : bubble sort (parent) + insertion sort (child)	CPU scheduling - Non-preemptive SJF
24	CPU scheduling - Round Robin	<code>fork()</code> + <code>execve()</code> : binary search
25	Orphan process	Shell with 'search' command
26	CPU scheduling - Preemptive Priority	Page replacement - FIFO
27	Disk scheduling - SSTF	Page replacement - LRU (counter)
28	Disk scheduling - SCAN	Page replacement - MFU
29	<code>fork()</code> + <code>execve()</code> : binary search	CPU scheduling - FCFS
30	CPU scheduling - Round Robin with I/O wait	Page replacement - MFU

Notes: (1) Lower priority number = higher priority. (2) 'Next CPU burst' is generated randomly and displayed; scheduling uses the first burst as the slips specify. (3) `nice(-5)` needs root (use `sudo`) - the program reports if it fails. (4) For `execve` slips compile `binsearch.c` first: `gcc binsearch.c -o binsearch`.

Slip 1

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

- Q.1) Write a C Menu driven Program to implement following functionality
- Accept Available
 - Display Allocation, Max
 - Display the contents of need matrix
 - Display Available

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	2	3	2	9	7	5	3	3	2
P1	4	0	0	5	2	2			
P2	5	0	4	1	0	4			
P3	4	3	3	4	4	4			
P4	2	2	4	6	5	5			

[15 Marks]

- Q.2 Write a C program that behaves like a shell which displays the command

prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as

- \$ count c filename: It will display the number of characters in given file
- \$ count w filename: It will display the number of words in given file
- \$ count l filename: It will display the number of lines in given file

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - Banker's algorithm - menu driven

```
/* Banker's algorithm - menu driven (Accept Available, Display Alloc/Max, Need, Available) */
#include <stdio.h>
#define MAXP 10
#define MAXR 10
int n, m, alloc[MAXP][MAXR], mx[MAXP][MAXR], need[MAXP][MAXR], avail[MAXR];

void acceptMatrix(const char *nm, int a[MAXP][MAXR]) {
    printf("Enter %s matrix (%d x %d):\n", nm, n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
}

void displayAllocMax() {
    printf("\nProcess\tAllocation\tMax\n\t");
    for (int k = 0; k < 2; k++) {
        for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
        printf("\t");
    }
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) printf("%d ", alloc[i][j]);
        printf("\t\t");
        for (int j = 0; j < m; j++) printf("%d ", mx[i][j]);
        printf("\n");
    }
}

void findNeed() {
    int bad = 0;
    printf("\nNeed matrix (Max - Allocation):\nProcess\t");
    for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) {
            need[i][j] = mx[i][j] - alloc[i][j];
            if (need[i][j] < 0) bad = 1;
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
}
```

```

    }
    if (bad) printf("Warning: negative Need => Max < Allocation for some process (check input data)\n");
}
void displayAvail() {
    printf("\nAvailable: ");
    for (int j = 0; j < m; j++) printf("%c=%d ", 'A' + j, avail[j]);
    printf("\n");
}
int main() {
    int ch;
    printf("Enter number of processes: "); scanf("%d", &n);
    printf("Enter number of resource types: "); scanf("%d", &m);
    acceptMatrix("Allocation", alloc);
    acceptMatrix("Max", mx);
    do {
        printf("\n---- MENU ----\n1. Accept Available\n2. Display Allocation, Max\n"
            "3. Display contents of Need matrix\n4. Display Available\n5. Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("Enter Available (%d values): ", m);
                for (int j = 0; j < m; j++) scanf("%d", &avail[j]);
                break;
            case 2: displayAllocMax(); break;
            case 3: findNeed(); break;
            case 4: displayAvail(); break;
            case 5: break;
            default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation matrix (5 x 3):
2 3 2
4 0 0
5 0 4
4 3 3
2 2 4
Enter Max matrix (5 x 3):
9 7 5
5 2 2
1 0 4
4 4 4
6 5 5

```

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 1
Enter Available (3 values): 3 3 2

```

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 2

```

Process	Allocation			Max		
	A	B	C	A	B	C
P0	2	3	2	9	7	5
P1	4	0	0	5	2	2
P2	5	0	4	1	0	4
P3	4	3	3	4	4	4
P4	2	2	4	6	5	5

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 3

```

```

Need matrix (Max - Allocation):
Process A B C
P0      7 4 3

```

```

P1      1 2 2
P2      -4 0 0
P3       0 1 1
P4       4 3 1
Warning: negative Need => Max < Allocation for some process (check input data)

```

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 4

```

Available: A=3 B=3 C=2

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 4

```

Available: A=3 B=3 C=2

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 5

```

Note: the slip's second matrix gives $\text{Max} < \text{Allocation}$ for some rows, so Need comes out negative. The program flags this; the data is exactly as printed on the slip.

Q2 - Shell with 'count' command

```

/* Simple shell: prompt '$', tokenizes, executes via fork/exec. Extra command: count c|w|l filename */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
void count(char *opt, char *fname) {
    FILE *fp = fopen(fname, "r");
    if (!fp) { perror(fname); return; }
    int c, ch = 0, w = 0, l = 0, inword = 0;
    while ((c = fgetc(fp)) != EOF) {
        ch++;
        if (c == '\n') l++;
        if (c == ' ' || c == '\n' || c == '\t') inword = 0;
        else if (!inword) { inword = 1; w++; }
    }
    fclose(fp);
    if (!strcmp(opt, "c")) printf("Number of characters = %d\n", ch);
    else if (!strcmp(opt, "w")) printf("Number of words = %d\n", w);
    else if (!strcmp(opt, "l")) printf("Number of lines = %d\n", l);
    else printf("count: invalid option '%s' (use c, w or l)\n", opt);
}
int main() {
    char line[256], *args[20];
    while (1) {
        printf("$ "); fflush(stdout);
        if (!fgets(line, sizeof line, stdin)) break;
        int k = 0;
        for (char *t = strtok(line, " \t\n"); t && k < 19; t = strtok(NULL, " \t\n")) args[k++] = t; /* tokenize */
        args[k] = NULL;
        if (k == 0) continue;
        if (!strcmp(args[0], "exit")) break;
        if (!strcmp(args[0], "count")) {
            if (k != 3) printf("Usage: count c|w|l filename\n"); else count(args[1], args[2]);
            continue;
        }
        pid_t pid = fork();
        if (pid == 0) { /* execute other commands in a child */
            execvp(args[0], args); perror(args[0]); exit(1);
        }
        else if (pid > 0) wait(NULL);
        else perror("fork");
    }
    return 0;
}

```

Sample output (input shown as typed)

```
$ ls -d /tmp
/tmp
$ count c t.txt
Number of characters = 36
$ count w t.txt
Number of words = 7
$ count l t.txt
Number of lines = 3
$ exit
```

Sample file t.txt = 3 lines: 'hello world' / 'foo bar hello' / 'last line'.

Q3 - Oral/Viva: see viva questions at the end.

Slip 2

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks. Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit

[15 Marks]

Q.2 Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
55, 58, 39, 18, 90, 160, 150, 38, 184
Start Head Position: 50

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - Linked file allocation

```
/* Linked file allocation simulation */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAXB 200
#define MAXF 20
int n, bv[MAXB]; /* bit vector: 1 = allocated, 0 = free */

void init() {
    srand(time(NULL));
    printf("Enter number of disk blocks: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) bv[i] = (rand() % 100) < 25; /* ~25% blocks randomly allocated */
}

void showBV() {
    printf("\nBit vector (1 = allocated, 0 = free):\n");
    for (int i = 0; i < n; i++) printf("%d", bv[i]);
    printf("\nFree blocks: ");
    for (int i = 0; i < n; i++) if (!bv[i]) printf("%d ", i);
    printf("\n");
}

struct File { char name[20]; int start, len, used; int blk[MAXB]; } dir[MAXF];
int nf = 0, nextp[MAXB]; /* nextp[b] = pointer stored in block b (-1 = end) */

void create() {
    char nm[20]; int len, freec = 0;
    printf("Enter file name and number of blocks: "); scanf("%s %d", nm, &len);
    for (int i = 0; i < n; i++) if (!bv[i]) freec++;
    if (len > freec) { printf("Not enough free blocks (free = %d)\n", freec); return; }
    int got = 0, prev = -1;
    strcpy(dir[nf].name, nm); dir[nf].len = len; dir[nf].used = 1;
    while (got < len) { /* any free block can be used (scattered) */
        int b = rand() % n;
        if (bv[b]) continue;
        bv[b] = 1; dir[nf].blk[got++] = b; nextp[b] = -1;
        if (prev == -1) dir[nf].start = b; else nextp[prev] = b;
        prev = b;
    }
}
```

```

    nf++;
    printf("File '%s' created. Start block = %d\n", nm, dir[nf - 1].start);
}
void show() {
    printf("\nFile\tStart\tLength\tChain\n");
    for (int i = 0; i < nf; i++) if (dir[i].used) {
        printf("%s\t%d\t%d\t", dir[i].name, dir[i].start, dir[i].len);
        for (int b = dir[i].start; b != -1; b = nextp[b]) printf("%d -> ", b);
        printf("NULL\n");
    }
}
void del() {
    char nm[20];
    printf("Enter file name to delete: "); scanf("%s", nm);
    for (int i = 0; i < nf; i++)
        if (dir[i].used && !strcmp(dir[i].name, nm)) {
            for (int k = 0; k < dir[i].len; k++) bv[dir[i].blk[k]] = 0;
            dir[i].used = 0; printf("File '%s' deleted\n", nm); return;
        }
    printf("File not found\n");
}
int main() {
    int ch;
    init();
    do {
        printf("\n1. Show Bit Vector\n2. Create New File\n3. Show Directory\n4. Delete File\n5. Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) { case 1: showBV(); break; case 2: create(); break; case 3: show(); break; case 4: del(); break; }
    } while (ch != 5);
    return 0;
}

```

Sample output (input shown as typed)

Enter number of disk blocks: 20

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 2

Enter file name and number of blocks: f1 4

File 'f1' created. Start block = 0

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 2

Enter file name and number of blocks: f2 3

File 'f2' created. Start block = 5

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 3

File	Start	Length	Chain
f1	0	4	0 -> 2 -> 3 -> 18 -> NULL
f2	5	3	5 -> 17 -> 11 -> NULL

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 1

Bit vector (1 = allocated, 0 = free):

10111101001111000111

Free blocks: 1 6 8 9 14 15 16

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 4

Enter file name to delete: f1

File 'f1' deleted

1. Show Bit Vector

```
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 3
```

```
File      Start   Length Chain
f2        5       3      5 -> 17 -> 11 -> NULL
```

```
1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 5
```

Disk size n and the pre-allocated blocks are random, so your bit vector will differ.

Q2 - Disk scheduling - FCFS

```
/* Disk scheduling - FCFS */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, req[50], total = 0;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    for (int i = 0; i < n; i++) {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter total number of disk blocks: 200
Enter number of requests: 9
Enter request string: 55 58 39 18 90 160 150 38 184
Enter current head position: 50

Order of service: 50 -> 55 -> 58 -> 39 -> 18 -> 90 -> 160 -> 150 -> 38 -> 184
Total number of head movements = 458
```

Q3 - Oral/Viva: see viva questions at the end.

Slip 3

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

- a) Calculate and display the content of need matrix?
b) Is the system in safe state? If display the safe sequence.

[15 Marks]

Q.2 Write a simulation program for disk scheduling using SSTF algorithm.
Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
98, 183, 37, 122, 14, 124, 65
Start Head Position: 53

[15 Marks]

Q.3. Oral/Viva [05 Marks]

Q1 - Banker's algorithm - Need matrix & safe sequence

```
/* Banker's algorithm: Need matrix + safety algorithm (safe sequence) */
#include <stdio.h>
#define MAXP 10
#define MAXR 10
int main() {
    int n, m, alloc[MAXP][MAXR], mx[MAXP][MAXR], need[MAXP][MAXR];
    int avail[MAXR], work[MAXR], fin[MAXP] = {0}, seq[MAXP], cnt = 0;
    printf("Enter number of processes and resource types: ");
    scanf("%d %d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) scanf("%d", &mx[i][j]);
    printf("Enter Available:\n");
    for (int j = 0; j < m; j++) scanf("%d", &avail[j]);

    printf("\nNeed matrix:\n\t");
    for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) { need[i][j] = mx[i][j] - alloc[i][j]; printf("%d ", need[i][j]); }
        printf("\n");
    }
    for (int j = 0; j < m; j++) work[j] = avail[j];
    int found = 1;
    while (cnt < n && found) {
        found = 0;
        for (int i = 0; i < n; i++) {
            if (fin[i]) continue;
            int ok = 1;
            for (int j = 0; j < m; j++) if (need[i][j] > work[j]) { ok = 0; break; }
            if (ok) {
                for (int j = 0; j < m; j++) work[j] += alloc[i][j];
                fin[i] = 1; seq[cnt++] = i; found = 1;
            }
        }
    }
    if (cnt == n) {
```

```

        printf("\nSystem is in SAFE state.\nSafe sequence: ");
        for (int i = 0; i < n; i++) printf("P%d%s", seq[i], i < n - 1 ? " -> " : "\n");
    } else printf("\nSystem is NOT in a safe state (deadlock possible).\n");
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes and resource types: 5 4
Enter Allocation matrix:
0 0 1 2
1 0 0 0
1 3 5 4
0 6 3 2
0 0 1 4
Enter Max matrix:
0 0 1 2
1 7 5 0
2 3 5 6
0 6 5 2
0 6 5 6
Enter Available:
1 5 2 0

```

```

Need matrix:
      A B C D
P0    0 0 0 0
P1    0 7 5 0
P2    1 0 0 2
P3    0 0 2 0
P4    0 6 4 2

```

```

System is in SAFE state.
Safe sequence: P0 -> P2 -> P3 -> P4 -> P1

```

Q2 - Disk scheduling - SSTF

```

/* Disk scheduling - SSTF */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, req[50], done[50] = {0}, total = 0;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    for (int c = 0; c < n; c++) {
        int best = -1, bd = 1 << 30;
        for (int i = 0; i < n; i++)
            if (!done[i] && abs(req[i] - head) < bd) { bd = abs(req[i] - head); best = i; }
        done[best] = 1;
        total += bd;
        head = req[best];
        printf(" -> %d", head);
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 98 183 37 122 14 124 65
Enter current head position: 53

Order of service: 53 -> 65 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total number of head movements = 232

```

Q3 - Oral/Viva: see viva questions at the end.

Slip 4

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Implement the Menu driven Banker's algorithm for accepting Allocation, Max from user

- a) Accept Available
- b) Display Allocation, Max
- c) Find Need and display It,
- d) Display Available

Consider the system with 3 resources types A, B, and C with 7, 2, 6 instances respectively consider the following snapshot:

Process	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	4	0	0	5	2	2
P2	5	0	4	1	0	4
P3	4	3	3	4	4	4
P4	2	2	4	6	5	5

[15 Marks]

Q.2 Write a simulation program for disk scheduling using SCAN algorithm.

Accept total Number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
82, 170, 43, 140, 24, 16, 190, 65
Starting Head position= 50
Direction: Left

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - Banker's algorithm - menu driven

```
/* Banker's algorithm - menu driven (Accept Available, Display Alloc/Max, Need, Available) */
#include <stdio.h>
#define MAXP 10
#define MAXR 10
int n, m, alloc[MAXP][MAXR], mx[MAXP][MAXR], need[MAXP][MAXR], avail[MAXR];

void acceptMatrix(const char *nm, int a[MAXP][MAXR]) {
    printf("Enter %s matrix (%d x %d):\n", nm, n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
}

void displayAllocMax() {
    printf("\nProcess\tAllocation\tMax\n\t");
    for (int k = 0; k < 2; k++) {
        for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
        printf("\t");
    }
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) printf("%d ", alloc[i][j]);
        printf("\t\t");
        for (int j = 0; j < m; j++) printf("%d ", mx[i][j]);
        printf("\n");
    }
}

void findNeed() {
    int bad = 0;
    printf("\nNeed matrix (Max - Allocation):\nProcess\t");
    for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) {
            need[i][j] = mx[i][j] - alloc[i][j];
            if (need[i][j] < 0) bad = 1;
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
}
```

```

    }
    if (bad) printf("Warning: negative Need => Max < Allocation for some process (check input data)\n");
}
void displayAvail() {
    printf("\nAvailable: ");
    for (int j = 0; j < m; j++) printf("%c=%d ", 'A' + j, avail[j]);
    printf("\n");
}
int main() {
    int ch;
    printf("Enter number of processes: "); scanf("%d", &n);
    printf("Enter number of resource types: "); scanf("%d", &m);
    acceptMatrix("Allocation", alloc);
    acceptMatrix("Max", mx);
    do {
        printf("\n---- MENU ----\n1. Accept Available\n2. Display Allocation, Max\n"
            "3. Display contents of Need matrix\n4. Display Available\n5. Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("Enter Available (%d values): ", m);
                for (int j = 0; j < m; j++) scanf("%d", &avail[j]);
                break;
            case 2: displayAllocMax(); break;
            case 3: findNeed(); break;
            case 4: displayAvail(); break;
            case 5: break;
            default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation matrix (5 x 3):
0 1 0
4 0 0
5 0 4
4 3 3
2 2 4
Enter Max matrix (5 x 3):
0 0 0
5 2 2
1 0 4
4 4 4
6 5 5

```

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 1
Enter Available (3 values): 7 2 6

```

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 2

```

Process	Allocation			Max		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	4	0	0	5	2	2
P2	5	0	4	1	0	4
P3	4	3	3	4	4	4
P4	2	2	4	6	5	5

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 3

```

```

Need matrix (Max - Allocation):
Process A B C
P0      0 -1 0

```

```
P1      1 2 2
P2      -4 0 0
P3      0 1 1
P4      4 3 1
Warning: negative Need => Max < Allocation for some process (check input data)
```

```
---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 4
```

Available: A=7 B=2 C=6

```
---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 5
```

Note: the slip's second matrix gives Max < Allocation for some rows, so Need comes out negative. The program flags this; the data is exactly as printed on the slip.

Q2 - Disk scheduling - SCAN

```
/* Disk scheduling - SCAN (elevator): head goes to the disk end, then reverses */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, dir, req[50], total = 0, pos;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): "); scanf("%d", &dir);

    for (int i = 0; i < n - 1; i++) /* sort ascending */
        for (int j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) { int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t; }

    int k = 0;
    while (k < n && req[k] < head) k++; /* req[0..k-1] < head <= req[k..] */

    pos = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) { /* LEFT first */
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        if (k < n) { /* requests remain on right: touch end 0 and reverse */
            total += pos - 0; pos = 0; printf(" -> %d", pos);
            for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        }
    } else { /* RIGHT first */
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        if (k > 0) {
            total += (blocks - 1) - pos; pos = blocks - 1; printf(" -> %d", pos);
            for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        }
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter total number of disk blocks: 200
Enter number of requests: 8
Enter request string: 82 170 43 140 24 16 190 65
Enter current head position: 50
Enter direction (0 = Left, 1 = Right): 0

Order of service: 50 -> 43 -> 24 -> 16 -> 0 -> 65 -> 82 -> 140 -> 170 -> 190
Total number of head movements = 240
```

SCAN: moves left to block 0, then reverses.

Q3 - Oral/Viva: see viva questions at the end.

Slip 5

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the simulation program for demand paging and show the page scheduling total and number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.

Reference String: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6

[15 Marks]

Q2 Consider a system with 'n' processes and 'm' resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not. (Use resource request algorithm)

[15 Marks]

Q.2. Oral/Viva

[5 Marks]

Q1 - Page replacement - FIFO

```
/* Demand paging - FIFO page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int next = 0; /* index of oldest page */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = 0, fault = 0;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = 1; break; }
        if (!hit) { frame[next] = ref[i]; next = (next + 1) % n; faults++; fault = 1; }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 4 5 6 3 4 7 3 4 5 6 7 2 4 6
```

```
Page | Frames
3 :   3   -   -   <- page fault
4 :   3   4   -   <- page fault
5 :   3   4   5   <- page fault
6 :   6   4   5   <- page fault
3 :   6   3   5   <- page fault
4 :   6   3   4   <- page fault
7 :   7   3   4   <- page fault
3 :   7   3   4
4 :   7   3   4
```

```

5 : 7 5 4 <- page fault
6 : 7 5 6 <- page fault
7 : 7 5 6
2 : 2 5 6 <- page fault
4 : 2 4 6 <- page fault
6 : 2 4 6

```

Total number of page faults = 11

Q2 - Resource-Request algorithm

/* Resource-Request algorithm (Banker's): need matrix, safety check, grant request? */

```

#include <stdio.h>
#define MAXP 10
#define MAXR 10
int n, m, alloc[MAXP][MAXR], mx[MAXP][MAXR], need[MAXP][MAXR], avail[MAXR];

int isSafe(int *seq) {
    int work[MAXR], fin[MAXP] = {0}, cnt = 0, found = 1;
    for (int j = 0; j < m; j++) work[j] = avail[j];
    while (cnt < n && found) {
        found = 0;
        for (int i = 0; i < n; i++) {
            if (fin[i]) continue;
            int ok = 1;
            for (int j = 0; j < m; j++) if (need[i][j] > work[j]) { ok = 0; break; }
            if (ok) {
                for (int j = 0; j < m; j++) work[j] += alloc[i][j];
                fin[i] = 1; seq[cnt++] = i; found = 1;
            }
        }
    }
    return cnt == n;
}

int main() {
    int total[MAXR], seq[MAXP], mode, req[MAXR], p;
    printf("Enter number of processes and resource types: ");
    scanf("%d %d", &n, &m);
    printf("Give 1) total instances of each resource 2) Available directly: ");
    scanf("%d", &mode);
    printf("Enter %s (%d values): ", mode == 1 ? "total instances" : "Available", m);
    for (int j = 0; j < m; j++) scanf("%d", mode == 1 ? &total[j] : &avail[j]);
    printf("Enter Allocation matrix:\n");
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) scanf("%d", &mx[i][j]);

    if (mode == 1)
        for (int j = 0; j < m; j++) {
            avail[j] = total[j];
            for (int i = 0; i < n; i++) avail[j] -= alloc[i][j];
        }
    printf("\nNeed matrix:\n\t");
    for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) { need[i][j] = mx[i][j] - alloc[i][j]; printf("%d ", need[i][j]); }
        printf("\n");
    }
    printf("Available: ");
    for (int j = 0; j < m; j++) printf("%d ", avail[j]);
    printf("\n");

    if (isSafe(seq)) {
        printf("\nCurrent state is SAFE. Safe sequence: ");
        for (int i = 0; i < n; i++) printf("P%d%s", seq[i], i < n - 1 ? " -> " : "\n");
    } else printf("\nCurrent state is NOT safe.\n");

    printf("\nEnter process number making request (0-%d): ", n - 1);
    scanf("%d", &p);
    printf("Enter request (%d values): ", m);
    for (int j = 0; j < m; j++) scanf("%d", &req[j]);

    for (int j = 0; j < m; j++)
        if (req[j] > need[p][j]) { printf("\nError: P%d exceeded its maximum claim. Request DENIED.\n", p); return 0; }
    for (int j = 0; j < m; j++)
        if (req[j] > avail[j]) { printf("\nResources not available. P%d must WAIT. Request cannot be granted immediately.\n", p); return 0; }
    for (int j = 0; j < m; j++) { avail[j] -= req[j]; alloc[p][j] += req[j]; need[p][j] -= req[j]; }
    if (isSafe(seq)) {
        printf("\nRequest can be GRANTED immediately. Safe sequence: ");
        for (int i = 0; i < n; i++) printf("P%d%s", seq[i], i < n - 1 ? " -> " : "\n");
    } else printf("\nGranting leaves system UNSAFE. Request cannot be granted immediately (P%d waits).\n", p);
    return 0;
}

```

Sample output (input shown as typed)

```
Enter number of processes and resource types: 5 3
Give 1) total instances of each resource 2) Available directly: 1
Enter total instances (3 values): 10 5 7
Enter Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Need matrix:
      A B C
P0     7 4 3
P1     1 2 2
P2     6 0 0
P3     0 1 1
P4     4 3 1
Available: 3 3 2
```

Current state is SAFE. Safe sequence: P1 -> P3 -> P4 -> P0 -> P2

```
Enter process number making request (0-4): 1
Enter request (3 values): 1 0 2
```

Request can be GRANTED immediately. Safe sequence: P1 -> P3 -> P4 -> P0 -> P2

No data on the slip; sample uses the standard textbook snapshot (total A,B,C = 10,5,7; P1 requests 1 0 2).

Q3 - Oral/Viva: see viva questions at the end.

Slip 6

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1. Write a program to simulate Sequential file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit

[15 Marks]

Q.2 Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. also display the total number of head movements (**Assume disk size=200**)
15, 45, 75, 105, 135, 165, 195, 80
Starting Head Position: 100
Direction: Right

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - Sequential file allocation

```
/* Sequential (contiguous) file allocation simulation */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAXB 200
#define MAXF 20
int n, bv[MAXB]; /* bit vector: 1 = allocated, 0 = free */

void init() {
    srand(time(NULL));
    printf("Enter number of disk blocks: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) bv[i] = (rand() % 100) < 25; /* ~25% blocks randomly allocated */
}

void showBV() {
    printf("\nBit vector (1 = allocated, 0 = free):\n");
    for (int i = 0; i < n; i++) printf("%d", bv[i]);
    printf("\nFree blocks: ");
    for (int i = 0; i < n; i++) if (!bv[i]) printf("%d ", i);
    printf("\n");
}

struct File { char name[20]; int start, len, used; } dir[MAXF];
int nf = 0;

void create() {
    char nm[20]; int len;
    printf("Enter file name and number of blocks: "); scanf("%s %d", nm, &len);
    for (int s = 0; s + len <= n; s++) { /* first-fit: find 'len' contiguous free blocks */
        int ok = 1;
        for (int k = 0; k < len; k++) if (bv[s + k]) { ok = 0; break; }
        if (ok) {
            for (int k = 0; k < len; k++) bv[s + k] = 1;
            strcpy(dir[nf].name, nm); dir[nf].start = s; dir[nf].len = len; dir[nf].used = 1; nf++;
            printf("File '%s' created: blocks %d to %d\n", nm, s, s + len - 1);
            return;
        }
    }
    printf("Not enough contiguous free blocks for '%s'\n", nm);
}
```

```

}
void show() {
    printf("\nFile\tStart\tLength\tBlocks\n");
    for (int i = 0; i < nf; i++) if (dir[i].used) {
        printf("%s\t%d\t%d\t", dir[i].name, dir[i].start, dir[i].len);
        for (int k = 0; k < dir[i].len; k++) printf("%d ", dir[i].start + k);
        printf("\n");
    }
}
void del() {
    char nm[20];
    printf("Enter file name to delete: "); scanf("%s", nm);
    for (int i = 0; i < nf; i++)
        if (dir[i].used && !strcmp(dir[i].name, nm)) {
            for (int k = 0; k < dir[i].len; k++) bv[dir[i].start + k] = 0;
            dir[i].used = 0; printf("File '%s' deleted\n", nm); return;
        }
    printf("File not found\n");
}
int main() {
    int ch;
    init();
    do {
        printf("\n1. Show Bit Vector\n2. Create New File\n3. Show Directory\n4. Delete File\n5. Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) { case 1: showBV(); break; case 2: create(); break; case 3: show(); break; case 4: del(); break; }
    } while (ch != 5);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of disk blocks: 20

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 2
Enter file name and number of blocks: f1 4
File 'f1' created: blocks 0 to 3

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 2
Enter file name and number of blocks: f2 3
File 'f2' created: blocks 5 to 7

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 2
Enter file name and number of blocks: f3 5
File 'f3' created: blocks 15 to 19

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 3

File      Start   Length  Blocks
f1         0         4       0 1 2 3
f2         5         3       5 6 7
f3        15         5      15 16 17 18 19

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 1

Bit vector (1 = allocated, 0 = free):
1111111110100111111
Free blocks: 9 11 12

1. Show Bit Vector
2. Create New File

```

```

3. Show Directory
4. Delete File
5. Exit
Choice: 4
Enter file name to delete: f2
File 'f2' deleted

```

```

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 3

```

File	Start	Length	Blocks
f1	0	4	0 1 2 3
f3	15	5	15 16 17 18 19

```

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 1

```

```

Bit vector (1 = allocated, 0 = free):
11111000101001111111
Free blocks: 5 6 7 9 11 12

```

```

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 5

```

Q2 - Disk scheduling - C-SCAN

```

/* Disk scheduling - C-SCAN: go to one end, jump to the other end, continue same direction */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, dir, req[50], total = 0, pos;
    printf("Enter total number of disk blocks (disk size): "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): "); scanf("%d", &dir);
    int last = blocks - 1;

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) { int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t; }

    int k = 0;
    while (k < n && req[k] < head) k++;

    pos = head;
    printf("\nOrder of service: %d", head);
    if (dir == 1) {
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        if (k > 0) {
            total += last - pos; pos = last; printf(" -> %d", pos); /* go to last block */
            total += last; pos = 0; printf(" -> %d", pos); /* jump to block 0 */
            for (int i = 0; i < k; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        }
    } else {
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        if (k < n) {
            total += pos; pos = 0; printf(" -> %d", pos); /* go to block 0 */
            total += last; pos = last; printf(" -> %d", pos); /* jump to last block */
            for (int i = n - 1; i >= k; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        }
    }
    printf("\nTotal number of head movements = %d (return jump counted)\n", total);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter total number of disk blocks (disk size): 200
Enter number of requests: 8
Enter request string: 15 45 75 105 135 165 195 80
Enter current head position: 100
Enter direction (0 = Left, 1 = Right): 1
Order of service: 100 -> 105 -> 135 -> 165 -> 195 -> 199 -> 0 -> 15 -> 45 -> 75 -> 80

```

Total number of head movements = 378 (return jump counted)

C-SCAN counts the jump from the last block (199) back to 0 as head movement.

Q3 - Oral/Viva: see viva questions at the end.

Slip 7

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Consider the following snapshot of the system.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1
P1	3	1	2	1	5	2	5	2				
P2	2	1	0	3	2	3	1	6				
P3	1	3	1	2	1	4	2	4				
P4	1	4	3	2	3	6	6	5				

Using Resource –Request algorithm to Check whether the current system is in safe state or not [15 Marks]

Q2. Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call. [15 Marks]

Q.3. Oral/Viva [05 Marks]

Q1 - Resource-Request algorithm

```
/* Resource-Request algorithm (Banker's): need matrix, safety check, grant request? */
#include <stdio.h>
#define MAXP 10
#define MAXR 10
int n, m, alloc[MAXP][MAXR], mx[MAXP][MAXR], need[MAXP][MAXR], avail[MAXR];

int isSafe(int *seq) {
    int work[MAXR], fin[MAXP] = {0}, cnt = 0, found = 1;
    for (int j = 0; j < m; j++) work[j] = avail[j];
    while (cnt < n && found) {
        found = 0;
        for (int i = 0; i < n; i++) {
            if (fin[i]) continue;
            int ok = 1;
            for (int j = 0; j < m; j++) if (need[i][j] > work[j]) { ok = 0; break; }
            if (ok) {
                for (int j = 0; j < m; j++) work[j] += alloc[i][j];
                fin[i] = 1; seq[cnt++] = i; found = 1;
            }
        }
    }
    return cnt == n;
}

int main() {
    int total[MAXR], seq[MAXP], mode, req[MAXR], p;
    printf("Enter number of processes and resource types: ");
    scanf("%d %d", &n, &m);
    printf("Give 1) total instances of each resource 2) Available directly: ");
    scanf("%d", &mode);
    printf("Enter %s (%d values): ", mode == 1 ? "total instances" : "Available", m);
    for (int j = 0; j < m; j++) scanf("%d", mode == 1 ? &total[j] : &avail[j]);
    printf("Enter Allocation matrix:\n");
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) scanf("%d", &mx[i][j]);

    if (mode == 1)
        for (int j = 0; j < m; j++) {
            avail[j] = total[j];
        }
}
```

```

        for (int i = 0; i < n; i++) avail[j] -= alloc[i][j];
    }
    printf("\nNeed matrix:\n\t");
    for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) { need[i][j] = mx[i][j] - alloc[i][j]; printf("%d ", need[i][j]); }
        printf("\n");
    }
    printf("Available: ");
    for (int j = 0; j < m; j++) printf("%d ", avail[j]);
    printf("\n");

    if (isSafe(seq)) {
        printf("\nCurrent state is SAFE. Safe sequence: ");
        for (int i = 0; i < n; i++) printf("P%d%s", seq[i], i < n - 1 ? " -> " : "\n");
    } else printf("\nCurrent state is NOT safe.\n");

    printf("\nEnter process number making request (0-%d): ", n - 1);
    scanf("%d", &p);
    printf("Enter request (%d values): ", m);
    for (int j = 0; j < m; j++) scanf("%d", &req[j]);

    for (int j = 0; j < m; j++)
        if (req[j] > need[p][j]) { printf("\nError: P%d exceeded its maximum claim. Request DENIED.\n", p); return 0; }
    for (int j = 0; j < m; j++)
        if (req[j] > avail[j]) { printf("\nResources not available. P%d must WAIT. Request cannot be granted immediately.\n", p); return 0; }
    for (int j = 0; j < m; j++) { avail[j] -= req[j]; alloc[p][j] += req[j]; need[p][j] -= req[j]; }
    if (isSafe(seq)) {
        printf("\nRequest can be GRANTED immediately. Safe sequence: ");
        for (int i = 0; i < n; i++) printf("P%d%s", seq[i], i < n - 1 ? " -> " : "\n");
    } else printf("\nGranting leaves system UNSAFE. Request cannot be granted immediately (P%d waits).\n", p);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes and resource types: 5 4
Give 1) total instances of each resource  2) Available directly: 2
Enter Available (4 values): 3 3 2 1
Enter Allocation matrix:
2 0 0 1
3 1 2 1
2 1 0 3
1 3 1 2
1 4 3 2
Enter Max matrix:
4 2 1 2
5 2 5 2
2 3 1 6
1 4 2 4
3 6 6 5

Need matrix:
  A B C D
P0  2 2 1 1
P1  2 1 3 1
P2  0 2 1 3
P3  0 1 1 2
P4  2 2 3 3
Available: 3 3 2 1

```

Current state is SAFE. Safe sequence: P0 -> P3 -> P4 -> P1 -> P2

```

Enter process number making request (0-4): 0
Enter request (4 values): 0 0 0 0

```

Request can be GRANTED immediately. Safe sequence: P0 -> P3 -> P4 -> P1 -> P2

Options: 2 = Available given directly. Safety result is shown first; request prompt after it can be 0 0 0 0.

Q2 - nice() system call with fork()

```

/* nice(): after fork(), give the child a higher priority (lower nice value) */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>
#include <sys/resource.h>
int main() {
    setvbuf(stdout, NULL, _IONBF, 0); /* unbuffered: output appears in real order */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) { /* child */

```

```

    printf("Child : initial nice value = %d\n", getpriority(PRIO_PROCESS, 0));
    errno = 0;
    int r = nice(-5);
    if (r == -1 && errno != 0) {
        perror("Child : nice(-5) failed (run with sudo for higher priority)");
    } else printf("Child : nice(-5) called, new nice value = %d (higher priority)\n", r);
    for (int i = 1; i <= 3; i++) { printf("Child : working %d (PID %d)\n", i, getpid()); sleep(1); }
    exit(0);
} else {
    printf("Parent: nice value = %d (unchanged)\n", getpriority(PRIO_PROCESS, 0));
    for (int i = 1; i <= 3; i++) { printf("Parent: working %d (PID %d)\n", i, getpid()); sleep(1); }
    wait(NULL);
    printf("Parent: child finished.\n");
}
return 0;
}

```

Sample output (input shown as typed)

```

Child : initial nice value = 0
Child : nice(-5) called, new nice value = -5 (higher priority)
Child : working 1 (PID 561)
Parent: nice value = 0 (unchanged)
Parent: working 1 (PID 560)
Child : working 2 (PID 561)
Parent: working 2 (PID 560)
Child : working 3 (PID 561)
Parent: working 3 (PID 560)
Parent: child finished.

```

Q3 - Oral/Viva: see viva questions at the end.

Slip 8

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a program to simulate Index file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned above and implement each option

- Show Bit Vector
- Create New File
- Show Directory
- Exit

[15 Marks]

Q.2 Write a program that demonstrates the use of nice () system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - Indexed file allocation

```
/* Indexed file allocation simulation */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAXB 200
#define MAXF 20
int n, bv[MAXB]; /* bit vector: 1 = allocated, 0 = free */

void init() {
    srand(time(NULL));
    printf("Enter number of disk blocks: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) bv[i] = (rand() % 100) < 25; /* ~25% blocks randomly allocated */
}

void showBV() {
    printf("\nBit vector (1 = allocated, 0 = free):\n");
    for (int i = 0; i < n; i++) printf("%d", bv[i]);
    printf("\nFree blocks: ");
    for (int i = 0; i < n; i++) if (!bv[i]) printf("%d ", i);
    printf("\n");
}

struct File { char name[20]; int index, len, used; int blk[MAXB]; } dir[MAXF];
int nf = 0;

int freeBlock() { int b; do b = rand() % n; while (bv[b]); return b; }

void create() {
    char nm[20]; int len, freec = 0;
    printf("Enter file name and number of data blocks: "); scanf("%s %d", nm, &len);
    for (int i = 0; i < n; i++) if (!bv[i]) freec++;
    if (len + 1 > freec) { printf("Not enough free blocks (need %d incl. index block, free = %d)\n", len + 1, freec); return; }
    int ib = freeBlock(); bv[ib] = 1; /* index block */
    strcpy(dir[nf].name, nm); dir[nf].index = ib; dir[nf].len = len; dir[nf].used = 1;
    for (int k = 0; k < len; k++) { int b = freeBlock(); bv[b] = 1; dir[nf].blk[k] = b; }
    nf++;
    printf("File '%s' created. Index block = %d\n", nm, ib);
}

void show() {
    printf("\nFile\tIndex\tLength\tBlocks listed in index\n");
    for (int i = 0; i < nf; i++) if (dir[i].used) {
```

```

        printf("%s\t%d\t%d\t", dir[i].name, dir[i].index, dir[i].len);
        for (int k = 0; k < dir[i].len; k++) printf("%d ", dir[i].blk[k]);
        printf("\n");
    }
}
void del() {
    char nm[20];
    printf("Enter file name to delete: "); scanf("%s", nm);
    for (int i = 0; i < nf; i++)
        if (dir[i].used && !strcmp(dir[i].name, nm)) {
            for (int k = 0; k < dir[i].len; k++) bv[dir[i].blk[k]] = 0;
            bv[dir[i].index] = 0; dir[i].used = 0; printf("File '%s' deleted\n", nm); return;
        }
    printf("File not found\n");
}
int main() {
    int ch;
    init();
    do {
        printf("\n1. Show Bit Vector\n2. Create New File\n3. Show Directory\n4. Delete File\n5. Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) { case 1: showBV(); break; case 2: create(); break; case 3: show(); break; case 4: del(); break; }
    } while (ch != 5);
    return 0;
}

```

Sample output (input shown as typed)

Enter number of disk blocks: 20

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 2

Enter file name and number of data blocks: f1 4

File 'f1' created. Index block = 11

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 2

Enter file name and number of data blocks: f2 3

File 'f2' created. Index block = 18

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 3

File	Index	Length	Blocks listed in index
f1	11	4	5 4 12 0
f2	18	3	17 14 13

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 1

Bit vector (1 = allocated, 0 = free):

11101110001111110111

Free blocks: 3 7 8 9 16

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 4

Enter file name to delete: f1

File 'f1' deleted

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 3

File	Index	Length	Blocks listed in index
------	-------	--------	------------------------

f2 18 3 17 14 13

1. Show Bit Vector
 2. Create New File
 3. Show Directory
 4. Delete File
 5. Exit
- Choice: 5

Q2 - nice() system call with fork()

```
/* nice(): after fork(), give the child a higher priority (lower nice value) */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>
#include <sys/resource.h>
int main() {
    setvbuf(stdout, NULL, _IONBF, 0);          /* unbuffered: output appears in real order */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {                             /* child */
        printf("Child : initial nice value = %d\n", getpriority(PRIO_PROCESS, 0));
        errno = 0;
        int r = nice(-5);                      /* negative => higher priority (needs root) */
        if (r == -1 && errno != 0) {
            perror("Child : nice(-5) failed (run with sudo for higher priority)");
        } else printf("Child : nice(-5) called, new nice value = %d (higher priority)\n", r);
        for (int i = 1; i <= 3; i++) { printf("Child : working %d (PID %d)\n", i, getpid()); sleep(1); }
        exit(0);
    } else {                                     /* parent */
        printf("Parent: nice value = %d (unchanged)\n", getpriority(PRIO_PROCESS, 0));
        for (int i = 1; i <= 3; i++) { printf("Parent: working %d (PID %d)\n", i, getpid()); sleep(1); }
        wait(NULL);
        printf("Parent: child finished.\n");
    }
    return 0;
}
```

Sample output (input shown as typed)

```
Child : initial nice value = 0
Child : nice(-5) called, new nice value = -5 (higher priority)
Child : working 1 (PID 561)
Parent: nice value = 0 (unchanged)
Parent: working 1 (PID 560)
Child : working 2 (PID 561)
Parent: working 2 (PID 560)
Child : working 3 (PID 561)
Parent: working 3 (PID 560)
Parent: child finished.
```

Q3 - Oral/Viva: see viva questions at the end.

Slip 9

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of 'n' frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

[15 Marks]

Q.2 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. the arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

[15 marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - Page replacement - Optimal

```
/* Demand paging - Optimal page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = 0, fault = 0;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = 1; break; }
        if (!hit) {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; } /* free frame */
            if (victim == -1) {
                int far = -1;
                for (int j = 0; j < n; j++) {
                    int k;
                    for (k = i + 1; k < len; k++) if (ref[k] == frame[j]) break; /* next use */
                    if (k > far) { far = k; victim = j; } /* never used again => k = len */
                }
            }
            frame[victim] = ref[i]; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("\ns\n", fault ? "<- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2
```

```
Page | Frames
8 : 8 - - <- page fault
5 : 8 5 - <- page fault
7 : 8 5 7 <- page fault
8 : 8 5 7
5 : 8 5 7
```

```

7 : 8 5 7
2 : 2 5 7 <- page fault
3 : 3 5 7 <- page fault
7 : 3 5 7
3 : 3 5 7
5 : 3 5 7
9 : 9 5 7 <- page fault
4 : 4 5 7 <- page fault
6 : 6 5 7 <- page fault
2 : 2 5 7 <- page fault

```

Total number of page faults = 9

Q2 - CPU scheduling - Non-preemptive SJF

```

/* Non-preemptive SJF CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time = %.2f\n", st / n, sw / n);
}

int main() {
    int done[MAXP] = {0}, t = 0;
    accept(0);
    for (int c = 0; c < n; c++) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (!done[i] && at[i] <= t && (p == -1 || bt[i] < bt[p])) p = i; /* shortest arrived job */
        if (p == -1) { /* nobody arrived: jump to next arrival */
            int nxt = -1;
            for (int i = 0; i < n; i++) if (!done[i] && (nxt == -1 || at[i] < at[nxt])) nxt = i;
            t = at[nxt]; c--; continue;
        }
        for (int k = 0; k < bt[p]; k++) tl[t + k] = p;
        t += bt[p]; ct[p] = t; done[p] = 1;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst: 0 7
P2 arrival time, first CPU burst: 2 4
P3 arrival time, first CPU burst: 4 1
P4 arrival time, first CPU burst: 5 4

```

Next CPU burst (random): P1=6 P2=5 P3=8 P4=7

Gantt chart:

| P1 | P3 | P2 | P4 |
0 7 8 12 16

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

Average Turnaround Time = 8.00

Average Waiting Time = 4.00

Q3 - Oral/Viva: see viva questions at the end.

Slip 10

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15Marks]

Q.2 Write a simulation program for disk scheduling using Look algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments. 55, 58, 39, 18, 90, 160, 150, 38.
Start Head Position: 100
Direction: Left [15 Marks]

Q.3. Oral/Viva [5 marks]

Q1 - CPU scheduling - Round Robin

```
/* Round Robin CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
    }
}
```

```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time    = %.2f\n", st / n, sw / n);
}

int main() {
    int rem[MAXP], q, queue[MAXT], f = 0, r = 0, inq[MAXP] = {0}, t = 0, left;
    accept(0);
    printf("Enter time quantum: "); scanf("%d", &q);
    for (int i = 0; i < n; i++) rem[i] = bt[i];
    left = n;
    while (left > 0) {
        for (int i = 0; i < n; i++) /* admit newly arrived processes */
            if (!inq[i] && at[i] <= t) { queue[r++] = i; inq[i] = 1; }
        if (f == r) { tl[t++] = -1; continue; } /* ready queue empty: CPU idle */
        int p = queue[f++];
        int run = rem[p] < q ? rem[p] : q;
        for (int k = 0; k < run; k++) {
            tl[t++] = p;
            for (int i = 0; i < n; i++) /* arrivals during this slice enter first */
                if (!inq[i] && at[i] <= t) { queue[r++] = i; inq[i] = 1; }
        }
        rem[p] -= run;
        if (rem[p] == 0) { ct[p] = t; left--; }
        else queue[r++] = p; /* preempted: back of queue */
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst: 0 5
P2 arrival time, first CPU burst: 1 3
P3 arrival time, first CPU burst: 2 8
P4 arrival time, first CPU burst: 3 6

Next CPU burst (random): P1=6 P2=5 P3=8 P4=7
Enter time quantum: 3

```

Gantt chart:

	P1	P2	P3	P4	P1	P3	P4	P3	
0	3	6	9	12	14	17	20	22	

Process	AT	BT	CT	TAT	WT
P1	0	5	14	14	9
P2	1	3	6	5	2
P3	2	8	22	20	12
P4	3	6	20	17	11

```

Average Turnaround Time = 14.00
Average Waiting Time    = 8.50

```

Q2 - Disk scheduling - LOOK

```

/* Disk scheduling - LOOK: head reverses at the last request, not at the disk end */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, dir, req[50], total = 0, pos;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): "); scanf("%d", &dir);

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) { int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t; }
    int k = 0;
    while (k < n && req[k] < head) k++;

    pos = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
    } else {
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}

```

```
}
```

Sample output (input shown as typed)

Enter total number of disk blocks: 200

Enter number of requests: 8

Enter request string: 55 58 39 18 90 160 150 38

Enter current head position: 100

Enter direction (0 = Left, 1 = Right): 0

Order of service: 100 -> 90 -> 58 -> 55 -> 39 -> 38 -> 18 -> 150 -> 160

Total number of head movements = 224

Q3 - Oral/Viva: see viva questions at the end.

Slip 11

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a C program to simulate Banker's algorithm for the purpose of Deadlock avoidance. The following snapshot of system, A, B, C and D are the resource type.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Implement the following Menu.

- Accept Available
 - Display Allocation, Max
 - Display the contents of need matrix
 - Display Available
- [15 Marks]

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the optimal page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2 [15 Marks]

Q.3. Oral/Viva [05 Marks]

Q1 - Banker's algorithm - menu driven

```
/* Banker's algorithm - menu driven (Accept Available, Display Alloc/Max, Need, Available) */
#include <stdio.h>
#define MAXP 10
#define MAXR 10
int n, m, alloc[MAXP][MAXR], mx[MAXP][MAXR], need[MAXP][MAXR], avail[MAXR];

void acceptMatrix(const char *nm, int a[MAXP][MAXR]) {
    printf("Enter %s matrix (%d x %d):\n", nm, n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
}

void displayAllocMax() {
    printf("\nProcess\tAllocation\tMax\n\t");
    for (int k = 0; k < 2; k++) {
        for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
        printf("\t");
    }
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) printf("%d ", alloc[i][j]);
        printf("\t\t");
        for (int j = 0; j < m; j++) printf("%d ", mx[i][j]);
        printf("\n");
    }
}

void findNeed() {
    int bad = 0;
    printf("\nNeed matrix (Max - Allocation):\nProcess\t");
    for (int j = 0; j < m; j++) printf("%c ", 'A' + j);
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t", i);
        for (int j = 0; j < m; j++) {
            need[i][j] = mx[i][j] - alloc[i][j];
            if (need[i][j] < 0) bad = 1;
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}
```

```

    }
    if (bad) printf("Warning: negative Need => Max < Allocation for some process (check input data)\n");
}
void displayAvail() {
    printf("\nAvailable: ");
    for (int j = 0; j < m; j++) printf("%c=%d ", 'A' + j, avail[j]);
    printf("\n");
}
int main() {
    int ch;
    printf("Enter number of processes: "); scanf("%d", &n);
    printf("Enter number of resource types: "); scanf("%d", &m);
    acceptMatrix("Allocation", alloc);
    acceptMatrix("Max", mx);
    do {
        printf("\n---- MENU ----\n1. Accept Available\n2. Display Allocation, Max\n"
            "3. Display contents of Need matrix\n4. Display Available\n5. Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("Enter Available (%d values): ", m);
                for (int j = 0; j < m; j++) scanf("%d", &avail[j]);
                break;
            case 2: displayAllocMax(); break;
            case 3: findNeed(); break;
            case 4: displayAvail(); break;
            case 5: break;
            default: printf("Invalid choice\n");
        }
    } while (ch != 5);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation matrix (5 x 3):
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Max matrix (5 x 3):
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

```

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 1
Enter Available (3 values): 0 0 0

```

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 2

```

Process	Allocation			Max		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	2	0	0	2	0	2
P2	3	0	3	0	0	0
P3	2	1	1	1	0	0
P4	0	0	2	0	0	2

```

---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 3

```

```

Need matrix (Max - Allocation):
Process A B C
P0      0 -1 0

```

```
P1      0 0 2
P2      -3 0 -3
P3      -1 -1 -1
P4      0 0 0
Warning: negative Need => Max < Allocation for some process (check input data)
```

```
---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 4
```

Available: A=0 B=0 C=0

```
---- MENU ----
1. Accept Available
2. Display Allocation, Max
3. Display contents of Need matrix
4. Display Available
5. Exit
Choice: 5
```

Note: the slip's second matrix gives Max < Allocation for some rows, so Need comes out negative. The program flags this; the data is exactly as printed on the slip.

Q2 - Page replacement - Optimal

```
/* Demand paging - Optimal page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = 0, fault = 0;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = 1; break; }
        if (!hit) {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; } /* free frame */
            if (victim == -1) {
                int far = -1;
                for (int j = 0; j < n; j++) {
                    int k;
                    for (k = i + 1; k < len; k++) if (ref[k] == frame[j]) break; /* next use */
                    if (k > far) { far = k; victim = j; } /* never used again => k = len */
                }
            }
            frame[victim] = ref[i]; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2
```

```
Page | Frames
8 : 8 - - <- page fault
5 : 8 5 - <- page fault
7 : 8 5 7 <- page fault
8 : 8 5 7
5 : 8 5 7
7 : 8 5 7
2 : 2 5 7 <- page fault
3 : 3 5 7 <- page fault
7 : 3 5 7
3 : 3 5 7
5 : 3 5 7
9 : 9 5 7 <- page fault
4 : 4 5 7 <- page fault
6 : 6 5 7 <- page fault
```

2 : 2 5 7 <- page fault

Total number of page faults = 9

Q3 - Oral/Viva: see viva questions at the end.

Slip 12

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate FCFS CPU scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 marks]

Q.2 Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call child process sorts the integers using insertion sort [15 marks]

Q.3. Oral/Viva [5 Marks]

Q1 - CPU scheduling - FCFS

```
/* FCFS CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time = %.2f\n", st / n, sw / n);
}

int main() {
    int done[MAXP] = {0}, t = 0;
    accept(0);
```

```

for (int c = 0; c < n; c++) {
    int p = -1; /* earliest arrival among unfinished */
    for (int i = 0; i < n; i++)
        if (!done[i] && (p == -1 || at[i] < at[p])) p = i;
    if (t < at[p]) t = at[p]; /* CPU idle until arrival */
    for (int k = 0; k < bt[p]; k++) tl[t + k] = p;
    t += bt[p]; ct[p] = t; done[p] = 1;
}
T = t;
report();
return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst: 0 5
P2 arrival time, first CPU burst: 1 3
P3 arrival time, first CPU burst: 2 8
P4 arrival time, first CPU burst: 3 6

Next CPU burst (random): P1=9 P2=10 P3=10 P4=6

```

Gantt chart:

```

| P1 | P2 | P3 | P4 |
0    5    8    16   22

```

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

```

Average Turnaround Time = 11.25
Average Waiting Time    = 5.75

```

Q2 - fork(): bubble sort (parent) + insertion sort (child)

```

/* fork(): parent bubble-sorts and waits; child insertion-sorts */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
void show(const char *who, int *a, int n) {
    printf("%s sorted array: ", who);
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
int main() {
    int n, a[50], b[50];
    printf("Enter number of integers: "); scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++) { scanf("%d", &a[i]); b[i] = a[i]; }

    fflush(stdout); /* avoid duplicated buffered output */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) { /* child: insertion sort */
        for (int i = 1; i < n; i++) {
            int key = b[i], j = i - 1;
            while (j >= 0 && b[j] > key) { b[j + 1] = b[j]; j--; }
            b[j + 1] = key;
        }
        printf("\n[Child PID %d] Insertion sort done\n", getpid());
        show("Child ", b, n);
        exit(0);
    } else { /* parent: bubble sort, then wait */
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
        printf("\n[Parent PID %d] Bubble sort done\n", getpid());
        show("Parent", a, n);
        wait(NULL); /* wait for child to finish */
        printf("[Parent] Child finished.\n");
    }
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of integers: 5
Enter 5 integers: 9 3 7 1 5

[Parent PID 569] Bubble sort done
Parent sorted array: 1 3 5 7 9
[Child PID 570] Insertion sort done

```

```
Child  sorted array: 1 3 5 7 9  
[Parent] Child finished.
```

Q3 - Oral/Viva: see viva questions at the end.

Slip 13

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q.2 Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. display the list of requests in the order in which it is served. Also display the total number of head moments
15, 30, 55, 90, 125, 140, 170
Starting Head Position: 100
Direction: Left

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - CPU scheduling - Non-preemptive SJF

```
/* Non-preemptive SJF CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
    }
}
```

```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time    = %.2f\n", st / n, sw / n);
}

int main() {
    int done[MAXP] = {0}, t = 0;
    accept(0);
    for (int c = 0; c < n; c++) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (!done[i] && at[i] <= t && (p == -1 || bt[i] < bt[p])) p = i; /* shortest arrived job */
        if (p == -1) { /* nobody arrived: jump to next arrival */
            int nxt = -1;
            for (int i = 0; i < n; i++) if (!done[i] && (nxt == -1 || at[i] < at[nxt])) nxt = i;
            t = at[nxt]; c--; continue;
        }
        for (int k = 0; k < bt[p]; k++) tl[t + k] = p;
        t += bt[p]; ct[p] = t; done[p] = 1;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst: 0 7
P2 arrival time, first CPU burst: 2 4
P3 arrival time, first CPU burst: 4 1
P4 arrival time, first CPU burst: 5 4

Next CPU burst (random): P1=6 P2=5 P3=8 P4=7

```

```

Gantt chart:
| P1 | P3 | P2 | P4 |
0   7   8   12  16

```

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

```

Average Turnaround Time = 8.00
Average Waiting Time    = 4.00

```

Q2 - Disk scheduling - LOOK

```

/* Disk scheduling - LOOK: head reverses at the last request, not at the disk end */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, dir, req[50], total = 0, pos;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): "); scanf("%d", &dir);

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) { int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t; }

    int k = 0;
    while (k < n && req[k] < head) k++;

    pos = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
    } else {
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 15 30 55 90 125 140 170

```

Enter current head position: 100
Enter direction (0 = Left, 1 = Right): 0

Order of service: 100 -> 90 -> 55 -> 30 -> 15 -> 125 -> 140 -> 170
Total number of head movements = 240

Q3 - Oral/Viva: see viva questions at the end.

Slip 14

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1. Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array.

[15 Marks]

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.

Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - fork() + execve(): binary search

Parent: `execve_parent.c`

```
/* Parent sorts array, child execve()s ./binsearch passing sorted array via command-line arguments */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int n, a[50], key;
    printf("Enter number of integers: "); scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter element to search: "); scanf("%d", &key);

    for (int i = 0; i < n - 1; i++) /* parent sorts the array */
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");

    fflush(stdout); /* avoid duplicated buffered output */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {
        char *args[60];
        char buf[60][12];
        args[0] = "./binsearch";
        sprintf(buf[0], "%d", key); args[1] = buf[0]; /* argv[1] = key */
        for (int i = 0; i < n; i++) { sprintf(buf[i + 1], "%d", a[i]); args[i + 2] = buf[i + 1]; }
        args[n + 2] = NULL;
        execve("./binsearch", args, NULL); /* load new program */
        perror("execve"); /* reached only on failure */
        exit(1);
    }
    wait(NULL);
    return 0;
}
```

Child program: `binsearch.c`

```

/* Child program loaded by execve: usage ./binsearch key sorted_elements... */
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    if (argc < 3) { printf("Usage: %s key sorted_array...\n", argv[0]); return 1; }
    int key = atoi(argv[1]), n = argc - 2, a[50];
    for (int i = 0; i < n; i++) a[i] = atoi(argv[i + 2]);
    int lo = 0, hi = n - 1, found = -1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] == key) { found = mid; break; }
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    if (found >= 0) printf("Binary search: %d found at position %d (index %d)\n", key, found + 1, found);
    else printf("Binary search: %d not found\n", key);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of integers: 6
Enter 6 integers: 9 3 7 1 5 11
Enter element to search: 7
Sorted array: 1 3 5 7 9 11
Binary search: 7 found at position 4 (index 3)

```

Q2 - Page replacement - FIFO

```

/* Demand paging - FIFO page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int next = 0; /* index of oldest page */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = 0, fault = 0;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = 1; break; }
        if (!hit) { frame[next] = ref[i]; next = (next + 1) % n; faults++; fault = 1; }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 4 5 6 3 4 7 3 4 5 6 7 2 4 6

```

```

Page | Frames
3 : 3 - - <- page fault
4 : 3 4 - <- page fault
5 : 3 4 5 <- page fault
6 : 6 4 5 <- page fault
3 : 6 3 5 <- page fault
4 : 6 3 4 <- page fault
7 : 7 3 4 <- page fault
3 : 7 3 4
4 : 7 3 4
5 : 7 5 4 <- page fault
6 : 7 5 6 <- page fault
7 : 7 5 6
2 : 2 5 6 <- page fault
4 : 2 4 6 <- page fault
6 : 2 4 6

```

Total number of page faults = 11

Q3 - Oral/Viva: see viva questions at the end.

Slip 15

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid) [15 Marks]

Q.2 Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time . [15 Marks]

Q.3. Oral/Viva [5 Marks]

Q1 - Orphan process

```
/* Orphan process: parent terminates before child finishes */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    setvbuf(stdout, NULL, _IONBF, 0); /* unbuffered: output appears in real order */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) { /* child */
        printf("Child : PID = %d, Parent PID = %d (before parent exits)\n", getpid(), getppid());
        sleep(5); /* parent dies meanwhile */
        printf("Child : I am now an ORPHAN. PID = %d, new Parent PID = %d (init/systemd adopted me)\n",
            getpid(), getppid());
    } else { /* parent */
        printf("Parent: PID = %d, created child PID = %d\n", getpid(), pid);
        sleep(1);
        printf("Parent: terminating before child finishes.\n");
        exit(0);
    }
    return 0;
}
```

Sample output (input shown as typed)

```
Child : PID = 575, Parent PID = 574 (before parent exits)
Parent: PID = 574, created child PID = 575
Parent: terminating before child finishes.
Child : I am now an ORPHAN. PID = 575, new Parent PID = 1 (init/systemd adopted me)
```

Q2 - CPU scheduling - Preemptive Priority

```
/* Preemptive Priority CPU scheduling (lower number = higher priority) */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
}
```

```

    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time = %.2f\n", st / n, sw / n);
}

int main() {
    int rem[MAXP], left, t = 0;
    accept(1);
    for (int i = 0; i < n; i++) rem[i] = bt[i];
    left = n;
    while (left > 0) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (rem[i] > 0 && at[i] <= t && (p == -1 || pr[i] < pr[p])) p = i; /* highest priority ready process */
        tl[t] = p;
        if (p != -1 && --rem[p] == 0) { ct[p] = t + 1; left--; }
        t++;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst, priority (low no. = high prio): 0 8 3
P2 arrival time, first CPU burst, priority (low no. = high prio): 1 4 2
P3 arrival time, first CPU burst, priority (low no. = high prio): 2 2 4
P4 arrival time, first CPU burst, priority (low no. = high prio): 3 1 1

```

Next CPU burst (random): P1=2 P2=6 P3=9 P4=3

Gantt chart:

```

| P1 | P2 | P4 | P2 | P1 | P3 |
0   1   3   4   6   13  15

```

Process	AT	BT	CT	TAT	WT
P1	0	8	13	13	5
P2	1	4	6	5	1
P3	2	2	15	13	11
P4	3	1	4	1	0

Average Turnaround Time = 8.00

Average Waiting Time = 4.25

Q3 - Oral/Viva: see viva questions at the end.

Slip 16

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

[15 Marks]

Q.2 Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as
a \$ count c filename: It will display the number of characters in given file
b \$ count w filename: It will display the number of words in given file
c \$ count l filename: It will display the number of lines in given file

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - Page replacement - LRU (counter)

```
/* Demand paging - LRU page replacement (counter method) */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int counter[10] = {0}, clock = 0; /* counter[j] = time page in frame j was last used */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = -1, fault = 0;
        clock++;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = j; break; }
        if (hit >= 0) counter[hit] = clock;
        else {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; }
            if (victim == -1) {
                victim = 0;
                for (int j = 1; j < n; j++) if (counter[j] < counter[victim]) victim = j; /* least recently used */
            }
            frame[victim] = ref[i]; counter[victim] = clock; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2
Page | Frames
```

```

3 : 3 - - <- page fault
5 : 3 5 - <- page fault
7 : 3 5 7 <- page fault
2 : 2 5 7 <- page fault
5 : 2 5 7
1 : 2 5 1 <- page fault
2 : 2 5 1
3 : 2 3 1 <- page fault
1 : 2 3 1
3 : 2 3 1
5 : 5 3 1 <- page fault
3 : 5 3 1
1 : 5 3 1
6 : 6 3 1 <- page fault
2 : 6 2 1 <- page fault

```

Total number of page faults = 9

Q2 - Shell with 'count' command

```

/* Simple shell: prompt '$', tokenizes, executes via fork/exec. Extra command: count c|w|l filename */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
void count(char *opt, char *fname) {
    FILE *fp = fopen(fname, "r");
    if (!fp) { perror(fname); return; }
    int c, ch = 0, w = 0, l = 0, inword = 0;
    while ((c = fgetc(fp)) != EOF) {
        ch++;
        if (c == '\n') l++;
        if (c == ' ' || c == '\n' || c == '\t') inword = 0;
        else if (!inword) { inword = 1; w++; }
    }
    fclose(fp);
    if (!strcmp(opt, "c")) printf("Number of characters = %d\n", ch);
    else if (!strcmp(opt, "w")) printf("Number of words = %d\n", w);
    else if (!strcmp(opt, "l")) printf("Number of lines = %d\n", l);
    else printf("count: invalid option '%s' (use c, w or l)\n", opt);
}
int main() {
    char line[256], *args[20];
    while (1) {
        printf("$ "); fflush(stdout);
        if (!fgets(line, sizeof line, stdin)) break;
        int k = 0;
        for (char *t = strtok(line, " \t\n"); t && k < 19; t = strtok(NULL, " \t\n")) args[k++] = t; /* tokenize */
        args[k] = NULL;
        if (k == 0) continue;
        if (!strcmp(args[0], "exit")) break;
        if (!strcmp(args[0], "count")) {
            if (k != 3) printf("Usage: count c|w|l filename\n"); else count(args[1], args[2]);
            continue;
        }
        pid_t pid = fork(); /* execute other commands in a child */
        if (pid == 0) { execvp(args[0], args); perror(args[0]); exit(1); }
        else if (pid > 0) wait(NULL);
        else perror("fork");
    }
    return 0;
}

```

Sample output (input shown as typed)

```

$ count l t.txt
Number of lines = 3
$ count w t.txt
Number of words = 7
$ count c t.txt
Number of characters = 36
$ exit

```

Q3 - Oral/Viva: see viva questions at the end.

Slip 17

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time [15 Marks]

Q.2 Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks. Write menu driven program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Delete File
- Exit

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - CPU scheduling - Non-preemptive SJF

```
/* Non-preemptive SJF CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
    }
}
```

```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time    = %.2f\n", st / n, sw / n);
}

int main() {
    int done[MAXP] = {0}, t = 0;
    accept(0);
    for (int c = 0; c < n; c++) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (!done[i] && at[i] <= t && (p == -1 || bt[i] < bt[p])) p = i; /* shortest arrived job */
        if (p == -1) { /* nobody arrived: jump to next arrival */
            int nxt = -1;
            for (int i = 0; i < n; i++) if (!done[i] && (nxt == -1 || at[i] < at[nxt])) nxt = i;
            t = at[nxt]; c--; continue;
        }
        for (int k = 0; k < bt[p]; k++) tl[t + k] = p;
        t += bt[p]; ct[p] = t; done[p] = 1;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst: 0 7
P2 arrival time, first CPU burst: 2 4
P3 arrival time, first CPU burst: 4 1
P4 arrival time, first CPU burst: 5 4

Next CPU burst (random): P1=6 P2=5 P3=8 P4=7

```

```

Gantt chart:
| P1 | P3 | P2 | P4 |
0   7   8   12  16

```

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

```

Average Turnaround Time = 8.00
Average Waiting Time    = 4.00

```

Q2 - Sequential file allocation

```

/* Sequential (contiguous) file allocation simulation */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAXB 200
#define MAXF 20
int n, bv[MAXB]; /* bit vector: 1 = allocated, 0 = free */

void init() {
    srand(time(NULL));
    printf("Enter number of disk blocks: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) bv[i] = (rand() % 100) < 25; /* ~25% blocks randomly allocated */
}

void showBV() {
    printf("\nBit vector (1 = allocated, 0 = free):\n");
    for (int i = 0; i < n; i++) printf("%d", bv[i]);
    printf("\nFree blocks: ");
    for (int i = 0; i < n; i++) if (!bv[i]) printf("%d ", i);
    printf("\n");
}

struct File { char name[20]; int start, len, used; } dir[MAXF];
int nf = 0;

void create() {
    char nm[20]; int len;
    printf("Enter file name and number of blocks: "); scanf("%s %d", nm, &len);
    for (int s = 0; s + len <= n; s++) { /* first-fit: find 'len' contiguous free blocks */
        int ok = 1;
        for (int k = 0; k < len; k++) if (bv[s + k]) { ok = 0; break; }
        if (ok) {
            for (int k = 0; k < len; k++) bv[s + k] = 1;
            strcpy(dir[nf].name, nm); dir[nf].start = s; dir[nf].len = len; dir[nf].used = 1; nf++;
            printf("File '%s' created: blocks %d to %d\n", nm, s, s + len - 1);
            return;
        }
    }
}

```

```

    }
}
printf("Not enough contiguous free blocks for '%s'\n", nm);
}
void show() {
printf("\nFile\tStart\tLength\tBlocks\n");
for (int i = 0; i < nf; i++) if (dir[i].used) {
printf("%s\t%d\t%d\t", dir[i].name, dir[i].start, dir[i].len);
for (int k = 0; k < dir[i].len; k++) printf("%d ", dir[i].start + k);
printf("\n");
}
}
void del() {
char nm[20];
printf("Enter file name to delete: "); scanf("%s", nm);
for (int i = 0; i < nf; i++)
if (dir[i].used && !strcmp(dir[i].name, nm)) {
for (int k = 0; k < dir[i].len; k++) bv[dir[i].start + k] = 0;
dir[i].used = 0; printf("File '%s' deleted\n", nm); return;
}
printf("File not found\n");
}
int main() {
int ch;
init();
do {
printf("\n1. Show Bit Vector\n2. Create New File\n3. Show Directory\n4. Delete File\n5. Exit\nChoice: ");
scanf("%d", &ch);
switch (ch) { case 1: showBV(); break; case 2: create(); break; case 3: show(); break; case 4: del(); break; }
} while (ch != 5);
return 0;
}

```

Sample output (input shown as typed)

```

Enter number of disk blocks: 20

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 2
Enter file name and number of blocks: f1 4
File 'f1' created: blocks 0 to 3

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 2
Enter file name and number of blocks: f2 3
File 'f2' created: blocks 5 to 7

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 2
Enter file name and number of blocks: f3 5
File 'f3' created: blocks 15 to 19

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 3

File   Start   Length  Blocks
f1      0        4       0 1 2 3
f2      5        3       5 6 7
f3     15        5     15 16 17 18 19

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 1

Bit vector (1 = allocated, 0 = free):
1111111110100111111
Free blocks: 9 11 12

```

```
1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 4
Enter file name to delete: f2
File 'f2' deleted
```

```
1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 3
```

File	Start	Length	Blocks
f1	0	4	0 1 2 3
f3	15	5	15 16 17 18 19

```
1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 1
```

```
Bit vector (1 = allocated, 0 = free):
11111000101001111111
Free blocks: 5 6 7 9 11 12
```

```
1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 5
```

Q3 - Oral/Viva: see viva questions at the end.

Slip 18

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1. Write the program to simulate Preemptive Shortest Job First (SJF) scheduling.
The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time [15 Marks]

Q.3 Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid()) [15 Marks]

Q.3. Oral/Viva [05 Marks]

Q1 - CPU scheduling - Preemptive SJF (SRTF)

```
/* Preemptive SJF (Shortest Remaining Time First) CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time = %.2f\n", st / n, sw / n);
}
```

```

int main() {
    int rem[MAXP], left, t = 0;
    accept(0);
    for (int i = 0; i < n; i++) rem[i] = bt[i];
    left = n;
    while (left > 0) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (rem[i] > 0 && at[i] <= t && (p == -1 || rem[i] < rem[p])) p = i;    /* least remaining time */
        tl[t] = p;
        /* -1 = idle */
        if (p != -1 && --rem[p] == 0) { ct[p] = t + 1; left--; }
        t++;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

Enter number of processes: 4
P1 arrival time, first CPU burst: 0 8
P2 arrival time, first CPU burst: 1 4
P3 arrival time, first CPU burst: 2 9
P4 arrival time, first CPU burst: 3 5

Next CPU burst (random): P1=2 P2=7 P3=5 P4=4

Gantt chart:

	P1	P2	P4	P1	P3	
0	1	5	10	17	26	

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Average Turnaround Time = 13.00
Average Waiting Time = 6.50

Q2 - Orphan process

```

/* Orphan process: parent terminates before child finishes */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    setvbuf(stdout, NULL, _IONBF, 0);    /* unbuffered: output appears in real order */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {    /* child */
        printf("Child : PID = %d, Parent PID = %d (before parent exits)\n", getpid(), getppid());
        sleep(5);    /* parent dies meanwhile */
        printf("Child : I am now an ORPHAN. PID = %d, new Parent PID = %d (init/systemd adopted me)\n",
            getpid(), getppid());
    } else {    /* parent */
        printf("Parent: PID = %d, created child PID = %d\n", getpid(), pid);
        sleep(1);
        printf("Parent: terminating before child finishes.\n");
        exit(0);
    }
    return 0;
}

```

Sample output (input shown as typed)

Child : PID = 575, Parent PID = 574 (before parent exits)
Parent: PID = 574, created child PID = 575
Parent: terminating before child finishes.
Child : I am now an ORPHAN. PID = 575, new Parent PID = 1 (init/systemd adopted me)

Q3 - Oral/Viva: see viva questions at the end.

Slip 19

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated. [15 Marks]

Q.2 Write a program that demonstrates the use of nice() system call.

After a child process is started using fork(), assign higher priority to the child using nice() system call [15 Marks]

Q.3. Oral/Viva [5 Marks]

Q1 - CPU scheduling - Preemptive Priority

```
/* Preemptive Priority CPU scheduling (lower number = higher priority) */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
}
```

```

    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time    = %.2f\n", st / n, sw / n);
}

int main() {
    int rem[MAXP], left, t = 0;
    accept(1);
    for (int i = 0; i < n; i++) rem[i] = bt[i];
    left = n;
    while (left > 0) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (rem[i] > 0 && at[i] <= t && (p == -1 || pr[i] < pr[p])) p = i;    /* highest priority ready process */
        tl[t] = p;
        if (p != -1 && --rem[p] == 0) { ct[p] = t + 1; left--; }
        t++;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst, priority (low no. = high prio): 0 8 3
P2 arrival time, first CPU burst, priority (low no. = high prio): 1 4 2
P3 arrival time, first CPU burst, priority (low no. = high prio): 2 2 4
P4 arrival time, first CPU burst, priority (low no. = high prio): 3 1 1

```

Next CPU burst (random): P1=2 P2=6 P3=9 P4=3

Gantt chart:

```

| P1 | P2 | P4 | P2 | P1 | P3 |
0   1   3   4   6   13  15

```

Process	AT	BT	CT	TAT	WT
P1	0	8	13	13	5
P2	1	4	6	5	1
P3	2	2	15	13	11
P4	3	1	4	1	0

Average Turnaround Time = 8.00

Average Waiting Time = 4.25

Q2 - nice() system call with fork()

```

/* nice(): after fork(), give the child a higher priority (lower nice value) */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>
#include <sys/resource.h>
int main() {
    setvbuf(stdout, NULL, _IONBF, 0);    /* unbuffered: output appears in real order */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {    /* child */
        printf("Child : initial nice value = %d\n", getpriority(PRIO_PROCESS, 0));
        errno = 0;
        int r = nice(-5);    /* negative => higher priority (needs root) */
        if (r == -1 && errno != 0) {
            perror("Child : nice(-5) failed (run with sudo for higher priority)");
        } else printf("Child : nice(-5) called, new nice value = %d (higher priority)\n", r);
        for (int i = 1; i <= 3; i++) { printf("Child : working %d (PID %d)\n", i, getpid()); sleep(1); }
        exit(0);
    } else {    /* parent */
        printf("Parent: nice value = %d (unchanged)\n", getpriority(PRIO_PROCESS, 0));
        for (int i = 1; i <= 3; i++) { printf("Parent: working %d (PID %d)\n", i, getpid()); sleep(1); }
        wait(NULL);
        printf("Parent: child finished.\n");
    }
    return 0;
}

```

Sample output (input shown as typed)

```

Child : initial nice value = 0
Child : nice(-5) called, new nice value = -5 (higher priority)
Child : working 1 (PID 561)
Parent: nice value = 0 (unchanged)
Parent: working 1 (PID 560)
Child : working 2 (PID 561)
Parent: working 2 (PID 560)
Child : working 3 (PID 561)

```

Parent: working 3 (PID 560)
Parent: child finished.

Q3 - Oral/Viva: see viva questions at the end.

Slip 20

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a simulation program for disk scheduling using LOOK algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

86, 147, 91, 177, 45, 12, 130

Starting Head Position: 60

Direction: Right

[15 Marks]

Q.2 Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - Disk scheduling - LOOK

```
/* Disk scheduling - LOOK: head reverses at the last request, not at the disk end */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, dir, req[50], total = 0, pos;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): "); scanf("%d", &dir);

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) { int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t; }

    int k = 0;
    while (k < n && req[k] < head) k++;

    pos = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) {
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
    } else {
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 86 147 91 177 45 12 130
Enter current head position: 60
Enter direction (0 = Left, 1 = Right): 1
```

Order of service: 60 -> 86 -> 91 -> 130 -> 147 -> 177 -> 45 -> 12

Total number of head movements = 282

Q2 - Page replacement - LRU (counter)

```
/* Demand paging - LRU page replacement (counter method) */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int counter[10] = {0}, clock = 0; /* counter[j] = time page in frame j was last used */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = -1, fault = 0;
        clock++;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = j; break; }
        if (hit >= 0) counter[hit] = clock;
        else {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; }
            if (victim == -1) {
                victim = 0;
                for (int j = 1; j < n; j++) if (counter[j] < counter[victim]) victim = j; /* least recently used */
            }
            frame[victim] = ref[i]; counter[victim] = clock; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2
```

```
Page | Frames
3 : 3 - - <- page fault
5 : 3 5 - <- page fault
7 : 3 5 7 <- page fault
2 : 2 5 7 <- page fault
5 : 2 5 7
1 : 2 5 1 <- page fault
2 : 2 5 1
3 : 2 3 1 <- page fault
1 : 2 3 1
3 : 2 3 1
5 : 5 3 1 <- page fault
3 : 5 3 1
1 : 5 3 1
6 : 6 3 1 <- page fault
2 : 6 2 1 <- page fault
```

Total number of page faults = 9

Q3 - Oral/Viva: see viva questions at the end.

Slip 21

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2 [15 Marks]

Q.2 Write a program that demonstrates the use of nice() system call.

After a child process is started using fork(), assign higher priority to the child using nice() system call. [15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - Page replacement - MFU

```
/* Demand paging - MFU (Most Frequently Used) page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int freq[10] = {0}, loaded[10] = {0}, clock = 0; /* freq: uses since loaded, loaded: load time (tie-break FIFO) */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = -1, fault = 0;
        clock++;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = j; break; }
        if (hit >= 0) freq[hit]++;
        else {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; }
            if (victim == -1) {
                victim = 0;
                for (int j = 1; j < n; j++)
                    if (freq[j] > freq[victim] || (freq[j] == freq[victim] && loaded[j] < loaded[victim])) victim = j;
            }
            frame[victim] = ref[i]; freq[victim] = 1; loaded[victim] = clock; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2
```

```
Page | Frames
8 : 8 - - <- page fault
5 : 8 5 - <- page fault
```

```

7 : 8 5 7 <- page fault
8 : 8 5 7
5 : 8 5 7
7 : 8 5 7
2 : 2 5 7 <- page fault
3 : 2 3 7 <- page fault
7 : 2 3 7
3 : 2 3 7
5 : 2 3 5 <- page fault
9 : 2 9 5 <- page fault
4 : 4 9 5 <- page fault
6 : 4 9 6 <- page fault
2 : 4 2 6 <- page fault

```

Total number of page faults = 10

Q2 - nice() system call with fork()

```

/* nice(): after fork(), give the child a higher priority (lower nice value) */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>
#include <sys/resource.h>
int main() {
    setvbuf(stdout, NULL, _IONBF, 0); /* unbuffered: output appears in real order */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) { /* child */
        printf("Child : initial nice value = %d\n", getpriority(PRIO_PROCESS, 0));
        errno = 0;
        int r = nice(-5); /* negative => higher priority (needs root) */
        if (r == -1 && errno != 0) {
            perror("Child : nice(-5) failed (run with sudo for higher priority)");
        } else printf("Child : nice(-5) called, new nice value = %d (higher priority)\n", r);
        for (int i = 1; i <= 3; i++) { printf("Child : working %d (PID %d)\n", i, getpid()); sleep(1); }
        exit(0);
    } else { /* parent */
        printf("Parent: nice value = %d (unchanged)\n", getpriority(PRIO_PROCESS, 0));
        for (int i = 1; i <= 3; i++) { printf("Parent: working %d (PID %d)\n", i, getpid()); sleep(1); }
        wait(NULL);
        printf("Parent: child finished.\n");
    }
    return 0;
}

```

Sample output (input shown as typed)

```

Child : initial nice value = 0
Child : nice(-5) called, new nice value = -5 (higher priority)
Child : working 1 (PID 561)
Parent: nice value = 0 (unchanged)
Parent: working 1 (PID 560)
Child : working 2 (PID 561)
Parent: working 2 (PID 560)
Child : working 3 (PID 561)
Parent: working 3 (PID 560)
Parent: child finished.

```

Q3 - Oral/Viva: see viva questions at the end.

Slip 22

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned below and implement each option.

- a. Show Bit Vector
- b. Create New File
- c. Show Directory
- d. Delete File
- e. Exit

[15 Marks]

Q.2 Write the program to simulate Non-preemptive Priority scheduling.

The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - Linked file allocation

```
/* Linked file allocation simulation */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAXB 200
#define MAXF 20
int n, bv[MAXB]; /* bit vector: 1 = allocated, 0 = free */

void init() {
    srand(time(NULL));
    printf("Enter number of disk blocks: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) bv[i] = (rand() % 100) < 25; /* ~25% blocks randomly allocated */
}

void showBV() {
    printf("\nBit vector (1 = allocated, 0 = free):\n");
    for (int i = 0; i < n; i++) printf("%d", bv[i]);
    printf("\nFree blocks: ");
    for (int i = 0; i < n; i++) if (!bv[i]) printf("%d ", i);
    printf("\n");
}

struct File { char name[20]; int start, len, used; int blk[MAXB]; } dir[MAXF];
int nf = 0, nextp[MAXB]; /* nextp[b] = pointer stored in block b (-1 = end) */

void create() {
    char nm[20]; int len, freec = 0;
    printf("Enter file name and number of blocks: "); scanf("%s %d", nm, &len);
    for (int i = 0; i < n; i++) if (!bv[i]) freec++;
    if (len > freec) { printf("Not enough free blocks (free = %d)\n", freec); return; }
    int got = 0, prev = -1;
    strcpy(dir[nf].name, nm); dir[nf].len = len; dir[nf].used = 1;
    while (got < len) { /* any free block can be used (scattered) */
        int b = rand() % n;
        if (bv[b]) continue;
        bv[b] = 1; dir[nf].blk[got++] = b; nextp[b] = -1;
        if (prev == -1) dir[nf].start = b; else nextp[prev] = b;
        prev = b;
    }
}
```

```

    nf++;
    printf("File '%s' created. Start block = %d\n", nm, dir[nf - 1].start);
}
void show() {
    printf("\nFile\tStart\tLength\tChain\n");
    for (int i = 0; i < nf; i++) if (dir[i].used) {
        printf("%s\t%d\t%d\t", dir[i].name, dir[i].start, dir[i].len);
        for (int b = dir[i].start; b != -1; b = nextp[b]) printf("%d -> ", b);
        printf("NULL\n");
    }
}
void del() {
    char nm[20];
    printf("Enter file name to delete: "); scanf("%s", nm);
    for (int i = 0; i < nf; i++)
        if (dir[i].used && !strcmp(dir[i].name, nm)) {
            for (int k = 0; k < dir[i].len; k++) bv[dir[i].blk[k]] = 0;
            dir[i].used = 0; printf("File '%s' deleted\n", nm); return;
        }
    printf("File not found\n");
}
int main() {
    int ch;
    init();
    do {
        printf("\n1. Show Bit Vector\n2. Create New File\n3. Show Directory\n4. Delete File\n5. Exit\nChoice: ");
        scanf("%d", &ch);
        switch (ch) { case 1: showBV(); break; case 2: create(); break; case 3: show(); break; case 4: del(); break; }
    } while (ch != 5);
    return 0;
}

```

Sample output (input shown as typed)

Enter number of disk blocks: 20

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 2

Enter file name and number of blocks: f1 4

File 'f1' created. Start block = 0

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 2

Enter file name and number of blocks: f2 3

File 'f2' created. Start block = 5

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 3

File	Start	Length	Chain
f1	0	4	0 -> 2 -> 3 -> 18 -> NULL
f2	5	3	5 -> 17 -> 11 -> NULL

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 1

Bit vector (1 = allocated, 0 = free):

10111101001111000111

Free blocks: 1 6 8 9 14 15 16

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit

Choice: 4

Enter file name to delete: f1

File 'f1' deleted

1. Show Bit Vector

```

2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 3

```

```

File      Start    Length Chain
f2        5        3        5 -> 17 -> 11 -> NULL

```

```

1. Show Bit Vector
2. Create New File
3. Show Directory
4. Delete File
5. Exit
Choice: 5

```

Q2 - CPU scheduling - Non-preemptive Priority

```

/* Non-preemptive Priority CPU scheduling (lower number = higher priority) */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time = %.2f\n", st / n, sw / n);
}

int main() {
    int done[MAXP] = {0}, t = 0;
    accept(1);
    for (int c = 0; c < n; c++) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (!done[i] && at[i] <= t && (p == -1 || pr[i] < pr[p])) p = i;
        if (p == -1) {
            int nxt = -1;
            for (int i = 0; i < n; i++) if (!done[i] && (nxt == -1 || at[i] < at[nxt])) nxt = i;
            t = at[nxt]; c--; continue;
        }
        for (int k = 0; k < bt[p]; k++) tl[t + k] = p;
        t += bt[p]; ct[p] = t; done[p] = 1;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 5
P1 arrival time, first CPU burst, priority (low no. = high prio): 0 10 3

```

P2 arrival time, first CPU burst, priority (low no. = high prio): 0 1 1
P3 arrival time, first CPU burst, priority (low no. = high prio): 0 2 4
P4 arrival time, first CPU burst, priority (low no. = high prio): 0 1 5
P5 arrival time, first CPU burst, priority (low no. = high prio): 0 5 2

Next CPU burst (random): P1=7 P2=1 P3=7 P4=8 P5=7

Gantt chart:

| P2 | P5 | P1 | P3 | P4 |
0 1 6 16 18 19

Process	AT	BT	CT	TAT	WT
P1	0	10	16	16	6
P2	0	1	1	1	0
P3	0	2	18	18	16
P4	0	1	19	19	18
P5	0	5	6	6	1

Average Turnaround Time = 12.00

Average Waiting Time = 8.20

Q3 - Oral/Viva: see viva questions at the end.

Slip 23

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call. Child process sorts the integers using insertion sort [15 Marks]

Q.2 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q.3. Oral/Viva [5 Marks]

Q1 - fork(): bubble sort (parent) + insertion sort (child)

```
/* fork(): parent bubble-sorts and waits; child insertion-sorts */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
void show(const char *who, int *a, int n) {
    printf("%s sorted array: ", who);
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
int main() {
    int n, a[50], b[50];
    printf("Enter number of integers: "); scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++) { scanf("%d", &a[i]); b[i] = a[i]; }

    fflush(stdout); /* avoid duplicated buffered output */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) { /* child: insertion sort */
        for (int i = 1; i < n; i++) {
            int key = b[i], j = i - 1;
            while (j >= 0 && b[j] > key) { b[j + 1] = b[j]; j--; }
            b[j + 1] = key;
        }
        printf("\n[Child PID %d] Insertion sort done\n", getpid());
        show("Child ", b, n);
        exit(0);
    } else { /* parent: bubble sort, then wait */
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
        printf("\n[Parent PID %d] Bubble sort done\n", getpid());
        show("Parent", a, n);
        wait(NULL); /* wait for child to finish */
        printf("[Parent] Child finished.\n");
    }
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of integers: 5
Enter 5 integers: 9 3 7 1 5
```

```
[Parent PID 569] Bubble sort done
Parent sorted array: 1 3 5 7 9
```

```
[Child PID 570] Insertion sort done
Child sorted array: 1 3 5 7 9
[Parent] Child finished.
```

Q2 - CPU scheduling - Non-preemptive SJF

```
/* Non-preemptive SJF CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time = %.2f\n", st / n, sw / n);
}

int main() {
    int done[MAXP] = {0}, t = 0;
    accept(0);
    for (int c = 0; c < n; c++) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (!done[i] && at[i] <= t && (p == -1 || bt[i] < bt[p])) p = i; /* shortest arrived job */
        if (p == -1) { /* nobody arrived: jump to next arrival */
            int nxt = -1;
            for (int i = 0; i < n; i++) if (!done[i] && (nxt == -1 || at[i] < at[nxt])) nxt = i;
            t = at[nxt]; c--; continue;
        }
        for (int k = 0; k < bt[p]; k++) tl[t + k] = p;
        t += bt[p]; ct[p] = t; done[p] = 1;
    }
    T = t;
    report();
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of processes: 4
P1 arrival time, first CPU burst: 0 7
P2 arrival time, first CPU burst: 2 4
P3 arrival time, first CPU burst: 4 1
P4 arrival time, first CPU burst: 5 4
Next CPU burst (random): P1=6 P2=5 P3=8 P4=7
```

Gantt chart:

| P1 | P3 | P2 | P4 |
0 7 8 12 16

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

Average Turnaround Time = 8.00

Average Waiting Time = 4.00

Q3 - Oral/Viva: see viva questions at the end.

Slip 24

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q.2 Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of execve() system call. The child process uses execve() system call to load new program that uses this sorted array for performing the binary search the particular item in the array. [15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - CPU scheduling - Round Robin

```
/* Round Robin CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
    }
}
```

```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time    = %.2f\n", st / n, sw / n);
}

int main() {
    int rem[MAXP], q, queue[MAXT], f = 0, r = 0, inq[MAXP] = {0}, t = 0, left;
    accept(0);
    printf("Enter time quantum: "); scanf("%d", &q);
    for (int i = 0; i < n; i++) rem[i] = bt[i];
    left = n;
    while (left > 0) {
        for (int i = 0; i < n; i++) /* admit newly arrived processes */
            if (!inq[i] && at[i] <= t) { queue[r++] = i; inq[i] = 1; }
        if (f == r) { tl[t++] = -1; continue; } /* ready queue empty: CPU idle */
        int p = queue[f++];
        int run = rem[p] < q ? rem[p] : q;
        for (int k = 0; k < run; k++) {
            tl[t++] = p;
            for (int i = 0; i < n; i++) /* arrivals during this slice enter first */
                if (!inq[i] && at[i] <= t) { queue[r++] = i; inq[i] = 1; }
        }
        rem[p] -= run;
        if (rem[p] == 0) { ct[p] = t; left--; }
        else queue[r++] = p; /* preempted: back of queue */
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 4
P1 arrival time, first CPU burst: 0 5
P2 arrival time, first CPU burst: 1 3
P3 arrival time, first CPU burst: 2 8
P4 arrival time, first CPU burst: 3 6

Next CPU burst (random): P1=6 P2=5 P3=8 P4=7
Enter time quantum: 3

```

Gantt chart:

	P1	P2	P3	P4	P1	P3	P4	P3	
0	3	6	9	12	14	17	20	22	

Process	AT	BT	CT	TAT	WT
P1	0	5	14	14	9
P2	1	3	6	5	2
P3	2	8	22	20	12
P4	3	6	20	17	11

```

Average Turnaround Time = 14.00
Average Waiting Time    = 8.50

```

Q2 - fork() + execve(): binary search

Parent: execve_parent.c

```

/* Parent sorts array, child execve()s ./binsearch passing sorted array via command-line arguments */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int n, a[50], key;
    printf("Enter number of integers: "); scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter element to search: "); scanf("%d", &key);

    for (int i = 0; i < n - 1; i++) /* parent sorts the array */
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");

    fflush(stdout); /* avoid duplicated buffered output */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {
        char *args[60];
        char buf[60][12];
        args[0] = "./binsearch";
        sprintf(buf[0], "%d", key); args[1] = buf[0]; /* argv[1] = key */
    }
}

```

```

        for (int i = 0; i < n; i++) { sprintf(buf[i + 1], "%d", a[i]); args[i + 2] = buf[i + 1]; }
        args[n + 2] = NULL;
        execve("./binsearch", args, NULL);          /* load new program */
        perror("execve");                          /* reached only on failure */
        exit(1);
    }
    wait(NULL);
    return 0;
}

```

Child program: binsearch.c

```

/* Child program loaded by execve: usage ./binsearch key sorted_elements... */
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    if (argc < 3) { printf("Usage: %s key sorted_array...\n", argv[0]); return 1; }
    int key = atoi(argv[1]), n = argc - 2, a[50];
    for (int i = 0; i < n; i++) a[i] = atoi(argv[i + 2]);
    int lo = 0, hi = n - 1, found = -1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] == key) { found = mid; break; }
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    if (found >= 0) printf("Binary search: %d found at position %d (index %d)\n", key, found + 1, found);
    else printf("Binary search: %d not found\n", key);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of integers: 6
Enter 6 integers: 9 3 7 1 5 11
Enter element to search: 7
Sorted array: 1 3 5 7 9 11
Binary search: 7 found at position 4 (index 3)

```

Q3 - Oral/Viva: see viva questions at the end.

Slip 25

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid())

[15 Marks]

Q.2 Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'search' as

- a. \$ search f filename pattern : It will search the first occurrence of pattern in the given file
- b. \$ search a filename pattern : It will search all the occurrence of pattern in the given file
- c. \$ search c filename pattern : It will count the number of occurrence of pattern in the given file

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - Orphan process

```
/* Orphan process: parent terminates before child finishes */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    setvbuf(stdout, NULL, _IONBF, 0); /* unbuffered: output appears in real order */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) { /* child */
        printf("Child : PID = %d, Parent PID = %d (before parent exits)\n", getpid(), getppid());
        sleep(5); /* parent dies meanwhile */
        printf("Child : I am now an ORPHAN. PID = %d, new Parent PID = %d (init/systemd adopted me)\n",
               getpid(), getppid());
    } else { /* parent */
        printf("Parent: PID = %d, created child PID = %d\n", getpid(), pid);
        sleep(1);
        printf("Parent: terminating before child finishes.\n");
        exit(0);
    }
    return 0;
}
```

Sample output (input shown as typed)

```
Child : PID = 575, Parent PID = 574 (before parent exits)
Parent: PID = 574, created child PID = 575
Parent: terminating before child finishes.
Child : I am now an ORPHAN. PID = 575, new Parent PID = 1 (init/systemd adopted me)
```

Q2 - Shell with 'search' command

```
/* Simple shell: prompt '$'. Extra command: search f|a|c filename pattern */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
void search(char *opt, char *fname, char *pat) {
    FILE *fp = fopen(fname, "r");
```

```

if (!fp) { perror(fname); return; }
char line[512]; int ln = 0, total = 0, found = 0;
while (fgets(line, sizeof line, fp)) {
    ln++;
    char *p = line; int here = 0;
    while ((p = strstr(p, pat)) != NULL) { here++; p += strlen(pat); } /* occurrences in this line */
    if (!here) continue;
    total += here;
    line[strcspn(line, "\n")] = 0;
    if (!strcmp(opt, "f")) { printf("First occurrence at line %d: %s\n", ln, line); found = 1; break; }
    if (!strcmp(opt, "a")) { printf("Line %d: %s\n", ln, line); found = 1; }
}
fclose(fp);
if (!strcmp(opt, "c")) printf("Pattern '%s' occurs %d time(s)\n", pat, total);
else if (!strcmp(opt, "f") || !strcmp(opt, "a")) { if (!found) printf("Pattern '%s' not found\n", pat); }
else printf("search: invalid option '%s' (use f, a or c)\n", opt);
}
int main() {
    char line[256], *args[20];
    while (1) {
        printf("$ "); fflush(stdout);
        if (!fgets(line, sizeof line, stdin)) break;
        int k = 0;
        for (char *t = strtok(line, " \t\n"); t && k < 19; t = strtok(NULL, " \t\n")) args[k++] = t;
        args[k] = NULL;
        if (k == 0) continue;
        if (!strcmp(args[0], "exit")) break;
        if (!strcmp(args[0], "search")) {
            if (k != 4) printf("Usage: search f|a|c filename pattern\n"); else search(args[1], args[2], args[3]);
            continue;
        }
        pid_t pid = fork();
        if (pid == 0) { execvp(args[0], args); perror(args[0]); exit(1); }
        else if (pid > 0) wait(NULL);
        else perror("fork");
    }
    return 0;
}

```

Sample output (input shown as typed)

```

$ search f t.txt hello
First occurrence at line 1: hello world
$ search a t.txt hello
Line 1: hello world
Line 2: foo bar hello
$ search c t.txt hello
Pattern 'hello' occurs 2 time(s)
$ exit

```

Q3 - Oral/Viva: see viva questions at the end.

Slip 26

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated

[15 Marks]

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.

Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - CPU scheduling - Preemptive Priority

```
/* Preemptive Priority CPU scheduling (lower number = higher priority) */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tat\tbt\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
}
```

```

    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time    = %.2f\n", st / n, sw / n);
}

int main() {
    int rem[MAXP], left, t = 0;
    accept(1);
    for (int i = 0; i < n; i++) rem[i] = bt[i];
    left = n;
    while (left > 0) {
        int p = -1;
        for (int i = 0; i < n; i++)
            if (rem[i] > 0 && at[i] <= t && (p == -1 || pr[i] < pr[p])) p = i;    /* highest priority ready process */
        tl[t] = p;
        if (p != -1 && --rem[p] == 0) { ct[p] = t + 1; left--; }
        t++;
    }
    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

Enter number of processes: 4
P1 arrival time, first CPU burst, priority (low no. = high prio): 0 8 3
P2 arrival time, first CPU burst, priority (low no. = high prio): 1 4 2
P3 arrival time, first CPU burst, priority (low no. = high prio): 2 2 4
P4 arrival time, first CPU burst, priority (low no. = high prio): 3 1 1

Next CPU burst (random): P1=2 P2=6 P3=9 P4=3

Gantt chart:

```

| P1 | P2 | P4 | P2 | P1 | P3 |
0    1    3    4    6    13   15

```

Process	AT	BT	CT	TAT	WT
P1	0	8	13	13	5
P2	1	4	6	5	1
P3	2	2	15	13	11
P4	3	1	4	1	0

Average Turnaround Time = 8.00

Average Waiting Time = 4.25

Q2 - Page replacement - FIFO

```

/* Demand paging - FIFO page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int next = 0;    /* index of oldest page */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = 0, fault = 0;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = 1; break; }
        if (!hit) { frame[next] = ref[i]; next = (next + 1) % n; faults++; fault = 1; }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}

```

Sample output (input shown as typed)

Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 4 5 6 3 4 7 3 4 5 6 7 2 4 6

Page	Frames
3 :	3 - - <- page fault
4 :	3 4 - <- page fault
5 :	3 4 5 <- page fault
6 :	6 4 5 <- page fault
3 :	6 3 5 <- page fault
4 :	6 3 4 <- page fault
7 :	7 3 4 <- page fault
3 :	7 3 4
4 :	7 3 4

```
5 : 7 5 4 <- page fault
6 : 7 5 6 <- page fault
7 : 7 5 6
2 : 2 5 6 <- page fault
4 : 2 4 6 <- page fault
6 : 2 4 6
```

Total number of page faults = 11

Q3 - Oral/Viva: see viva questions at the end.

Slip 27

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a simulation program for disk scheduling using SSTF algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

30, 10, 60, 95, 120, 150, 175

Start Head Position: 50

[15 Marks]

Q.2 Write the simulation program to implement demand paging

and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

[15 Marks]

Q.3. Oral/Viva

[05 Marks]

Q1 - Disk scheduling - SSTF

```
/* Disk scheduling - SSTF */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, req[50], done[50] = {0}, total = 0;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);

    printf("\nOrder of service: %d", head);
    for (int c = 0; c < n; c++) {
        int best = -1, bd = 1 << 30;
        for (int i = 0; i < n; i++)
            if (!done[i] && abs(req[i] - head) < bd) { bd = abs(req[i] - head); best = i; }
        done[best] = 1;
        total += bd;
        head = req[best];
        printf(" -> %d", head);
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter total number of disk blocks: 200
Enter number of requests: 7
Enter request string: 30 10 60 95 120 150 175
Enter current head position: 50
```

```
Order of service: 50 -> 60 -> 30 -> 10 -> 95 -> 120 -> 150 -> 175
Total number of head movements = 225
```

Q2 - Page replacement - LRU (counter)

```
/* Demand paging - LRU page replacement (counter method) */
#include <stdio.h>
```

```

#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int counter[10] = {0}, clock = 0; /* counter[j] = time page in frame j was last used */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = -1, fault = 0;
        clock++;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = j; break; }
        if (hit >= 0) counter[hit] = clock;
        else {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; }
            if (victim == -1) {
                victim = 0;
                for (int j = 1; j < n; j++) if (counter[j] < counter[victim]) victim = j; /* least recently used */
            }
            frame[victim] = ref[i]; counter[victim] = clock; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 3 5 7 2 5 1 2 3 1 3 5 3 1 6 2

```

```

Page | Frames
3 : 3 - - <- page fault
5 : 3 5 - <- page fault
7 : 3 5 7 <- page fault
2 : 2 5 7 <- page fault
5 : 2 5 7
1 : 2 5 1 <- page fault
2 : 2 5 1
3 : 2 3 1 <- page fault
1 : 2 3 1
3 : 2 3 1
5 : 5 3 1 <- page fault
3 : 5 3 1
1 : 5 3 1
6 : 6 3 1 <- page fault
2 : 6 2 1 <- page fault

```

Total number of page faults = 9

Q3 - Oral/Viva: see viva questions at the end.

Slip 28

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write a simulation program for disk scheduling using SCAN algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

10, 25, 75, 90, 130, 145, 180, 55

Starting Head position= 80

Direction: Left

[15 Marks]

Q.2 Write the simulation program for demand paging and show the page

scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

[15 Marks]

Q.3. Oral/Viva

[5 Marks]

Q1 - Disk scheduling - SCAN

```
/* Disk scheduling - SCAN (elevator): head goes to the disk end, then reverses */
#include <stdio.h>
#include <stdlib.h>
int main() {
    int blocks, n, head, dir, req[50], total = 0, pos;
    printf("Enter total number of disk blocks: "); scanf("%d", &blocks);
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request string: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter current head position: "); scanf("%d", &head);
    printf("Enter direction (0 = Left, 1 = Right): "); scanf("%d", &dir);

    for (int i = 0; i < n - 1; i++) /* sort ascending */
        for (int j = 0; j < n - i - 1; j++)
            if (req[j] > req[j + 1]) { int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t; }

    int k = 0;
    while (k < n && req[k] < head) k++; /* req[0..k-1] < head <= req[k..] */

    pos = head;
    printf("\nOrder of service: %d", head);
    if (dir == 0) { /* LEFT first */
        for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        if (k < n) { /* requests remain on right: touch end 0 and reverse */
            total += pos - 0; pos = 0; printf(" -> %d", pos);
            for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        }
    } else { /* RIGHT first */
        for (int i = k; i < n; i++) { total += req[i] - pos; pos = req[i]; printf(" -> %d", pos); }
        if (k > 0) {
            total += (blocks - 1) - pos; pos = blocks - 1; printf(" -> %d", pos);
            for (int i = k - 1; i >= 0; i--) { total += pos - req[i]; pos = req[i]; printf(" -> %d", pos); }
        }
    }
    printf("\nTotal number of head movements = %d\n", total);
    return 0;
}
```

Sample output (input shown as typed)

Enter total number of disk blocks: 200

```

Enter number of requests: 8
Enter request string: 10 25 75 90 130 145 180 55
Enter current head position: 80
Enter direction (0 = Left, 1 = Right): 0

Order of service: 80 -> 75 -> 55 -> 25 -> 10 -> 0 -> 90 -> 130 -> 145 -> 180
Total number of head movements = 260

```

Q2 - Page replacement - MFU

```

/* Demand paging - MFU (Most Frequently Used) page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int freq[10] = {0}, loaded[10] = {0}, clock = 0; /* freq: uses since loaded, loaded: load time (tie-break FIFO) */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = -1, fault = 0;
        clock++;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = j; break; }
        if (hit >= 0) freq[hit]++;
        else {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; }
            if (victim == -1) {
                victim = 0;
                for (int j = 1; j < n; j++)
                    if (freq[j] > freq[victim] || (freq[j] == freq[victim] && loaded[j] < loaded[victim])) victim = j;
            }
            frame[victim] = ref[i]; freq[victim] = 1; loaded[victim] = clock; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}

```

Sample output (input shown as typed)

```

Enter number of frames: 3
Enter length of reference string: 15
Enter reference string: 8 5 7 8 5 7 2 3 7 3 5 9 4 6 2

```

```

Page | Frames
8 : 8 - - <- page fault
5 : 8 5 - <- page fault
7 : 8 5 7 <- page fault
8 : 8 5 7
5 : 8 5 7
7 : 8 5 7
2 : 2 5 7 <- page fault
3 : 2 3 7 <- page fault
7 : 2 3 7
3 : 2 3 7
5 : 2 3 5 <- page fault
9 : 2 9 5 <- page fault
4 : 4 9 5 <- page fault
6 : 4 9 6 <- page fault
2 : 4 2 6 <- page fault

```

Total number of page faults = 10

Q3 - Oral/Viva: see viva questions at the end.

Slip 29

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array [15 Marks]

Q.2 Write the program to simulate FCFS CPU-scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time. [15 Marks]

Q.3. Oral/Viva [5 Marks]

Q1 - fork() + execve(): binary search

Parent: `execve_parent.c`

```
/* Parent sorts array, child execve()s ./binsearch passing sorted array via command-line arguments */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int n, a[50], key;
    printf("Enter number of integers: "); scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter element to search: "); scanf("%d", &key);

    for (int i = 0; i < n - 1; i++) /* parent sorts the array */
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; }
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");

    fflush(stdout); /* avoid duplicated buffered output */
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }
    if (pid == 0) {
        char *args[60];
        char buf[60][12];
        args[0] = "./binsearch";
        sprintf(buf[0], "%d", key); args[1] = buf[0]; /* argv[1] = key */
        for (int i = 0; i < n; i++) { sprintf(buf[i + 1], "%d", a[i]); args[i + 2] = buf[i + 1]; }
        args[n + 2] = NULL;
        execve("./binsearch", args, NULL); /* load new program */
        perror("execve"); /* reached only on failure */
        exit(1);
    }
    wait(NULL);
    return 0;
}
```

Child program: binsearch.c

```
/* Child program loaded by execve: usage ./binsearch key sorted_elements... */
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    if (argc < 3) { printf("Usage: %s key sorted_array...\n", argv[0]); return 1; }
    int key = atoi(argv[1]), n = argc - 2, a[50];
    for (int i = 0; i < n; i++) a[i] = atoi(argv[i + 2]);
    int lo = 0, hi = n - 1, found = -1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] == key) { found = mid; break; }
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    if (found >= 0) printf("Binary search: %d found at position %d (index %d)\n", key, found + 1, found);
    else printf("Binary search: %d not found\n", key);
    return 0;
}
```

Sample output (input shown as typed)

```
Enter number of integers: 6
Enter 6 integers: 9 3 7 1 5 11
Enter element to search: 7
Sorted array: 1 3 5 7 9 11
Binary search: 7 found at position 4 (index 3)
```

Q2 - CPU scheduling - FCFS

```
/* FCFS CPU scheduling */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 2000
int n, at[MAXP], bt[MAXP], nb[MAXP], pr[MAXP], ct[MAXP], tl[MAXT], T = 0;

void accept(int withPrio) {
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst%s: ", i + 1, withPrio ? ", priority (low no. = high prio)" : "");
        scanf("%d %d", &at[i], &bt[i]);
        if (withPrio) scanf("%d", &pr[i]);
        nb[i] = rand() % 10 + 1; /* next CPU burst generated randomly */
    }
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, nb[i]);
    printf("\n");
    for (int t = 0; t < MAXT; t++) tl[t] = -1;
}

void report() {
    float sw = 0, st = 0;
    printf("\nGantt chart:\n|");
    int s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) {
            if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1);
            s = t;
        }
    printf("\n0");
    s = 0;
    for (int t = 1; t <= T; t++)
        if (t == T || tl[t] != tl[s]) { printf(" %d", t); s = t; }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        st += tat; sw += wt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], tat, wt);
    }
    printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time = %.2f\n", st / n, sw / n);
}

int main() {
    int done[MAXP] = {0}, t = 0;
    accept(0);
    for (int c = 0; c < n; c++) {
        int p = -1; /* earliest arrival among unfinished */
        for (int i = 0; i < n; i++)
            if (!done[i] && (p == -1 || at[i] < at[p])) p = i;
        if (t < at[p]) t = at[p]; /* CPU idle until arrival */
        for (int k = 0; k < bt[p]; k++) tl[t + k] = p;
        t += bt[p]; ct[p] = t; done[p] = 1;
    }
}
```

```

    T = t;
    report();
    return 0;
}

```

Sample output (input shown as typed)

Enter number of processes: 4

P1 arrival time, first CPU burst: 0 5

P2 arrival time, first CPU burst: 1 3

P3 arrival time, first CPU burst: 2 8

P4 arrival time, first CPU burst: 3 6

Next CPU burst (random): P1=9 P2=10 P3=10 P4=6

Gantt chart:

```

| P1 | P2 | P3 | P4 |
0    5    8    16   22

```

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average Turnaround Time = 11.25

Average Waiting Time = 5.75

Q3 - Oral/Viva: see viva questions at the end.

Slip 30

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM–V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I

Duration: 3 Hours

Maximum Marks:35

Q.1 Write the program to simulate Round Robin (RR) CPU-scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

[15 Marks]

Q.2 Write the simulation program for demand paging and show the page scheduling and total number of page faults according to the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8

[15 Marks]

Q.3 Oral/Viva

[05 Marks]

Q1 - CPU scheduling - Round Robin with I/O wait

```
/* Round Robin with fixed I/O wait (2 units): CPU(first burst) -> I/O(2) -> CPU(next random burst) */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAXP 20
#define MAXT 3000
#define IOWAIT 2
int n, at[MAXP], b1[MAXP], b2[MAXP], ct[MAXP], tl[MAXT], T;

int main() {
    int rem[MAXP], phase[MAXP] = {0}, ioend[MAXP], state[MAXP] = {0}; /* state: 0 not yet, 1 ready, 2 in I/O, 3 done */
    int q, queue[MAXT], f = 0, r = 0, t = 0, left;
    srand(time(NULL));
    printf("Enter number of processes: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("P%d arrival time, first CPU burst: ", i + 1);
        scanf("%d %d", &at[i], &b1[i]);
        b2[i] = rand() % 10 + 1;
        rem[i] = b1[i];
    }
    printf("Enter time quantum: "); scanf("%d", &q);
    printf("\nNext CPU burst (random): ");
    for (int i = 0; i < n; i++) printf("P%d=%d ", i + 1, b2[i]);
    printf("\n");
    for (int i = 0; i < MAXT; i++) tl[i] = -1;
    left = n;
    while (left > 0) {
        for (int i = 0; i < n; i++) {
            if (state[i] == 0 && at[i] <= t) { state[i] = 1; queue[r++] = i; }
            if (state[i] == 2 && ioend[i] <= t) { state[i] = 1; queue[r++] = i; } /* I/O complete */
        }
        if (f == r) { tl[t++] = -1; continue; }
        int p = queue[f++];
        int run = rem[p] < q ? rem[p] : q;
        for (int k = 0; k < run; k++) {
            tl[t++] = p;
            for (int i = 0; i < n; i++) {
                if (state[i] == 0 && at[i] <= t) { state[i] = 1; queue[r++] = i; }
                if (state[i] == 2 && ioend[i] <= t) { state[i] = 1; queue[r++] = i; }
            }
        }
    }
}
```

```

    }
}
rem[p] -= run;
if (rem[p] > 0) queue[r++] = p;
else if (phase[p] == 0) { phase[p] = 1; rem[p] = b2[p]; state[p] = 2; ioend[p] = t + IOWAIT; }
else { state[p] = 3; ct[p] = t; left--; }
}
T = t;
printf("\nGantt chart (CPU):\n");
int s = 0;
for (int x = 1; x <= T; x++)
    if (x == T || tl[x] != tl[s]) { if (tl[s] < 0) printf(" Idle |"); else printf(" P%d |", tl[s] + 1); s = x; }
printf("\n0"); s = 0;
for (int x = 1; x <= T; x++) if (x == T || tl[x] != tl[s]) { printf("    %d", x); s = x; }
float st = 0, sw = 0;
printf("\n\nProcess\tAT\tBurst1\tBurst2\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    int tat = ct[i] - at[i], wt = tat - b1[i] - b2[i] - IOWAIT; /* waiting = TAT - CPU - I/O */
    st += tat; sw += wt;
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], b1[i], b2[i], ct[i], tat, wt);
}
printf("\nAverage Turnaround Time = %.2f\nAverage Waiting Time    = %.2f\n", st / n, sw / n);
return 0;
}

```

Sample output (input shown as typed)

```

Enter number of processes: 3
P1 arrival time, first CPU burst: 0 5
P2 arrival time, first CPU burst: 1 3
P3 arrival time, first CPU burst: 2 6
Enter time quantum: 2

```

Next CPU burst (random): P1=6 P2=7 P3=3

Gantt chart (CPU):

	P1	P2	P3	P1	P2	P3	P1	P2	P3	P1	P2	P3	P1	P2	P3	P1	P2
0	2	4	6	8	9	11	12	14	16	18	20	22	24	26	27	29	30

Process	AT	Burst1	Burst2	CT	TAT	WT
P1	0	5	6	29	29	16
P2	1	3	7	30	29	17
P3	2	6	3	27	25	14

```

Average Turnaround Time = 27.67
Average Waiting Time    = 15.67

```

Each process: CPU burst 1 -> fixed I/O wait of 2 units -> randomly generated CPU burst 2.

Q2 - Page replacement - MFU

```

/* Demand paging - MFU (Most Frequently Used) page replacement */
#include <stdio.h>
#define MAXN 50
int main() {
    int n, len, ref[MAXN], frame[10], faults = 0;
    printf("Enter number of frames: "); scanf("%d", &n);
    printf("Enter length of reference string: "); scanf("%d", &len);
    printf("Enter reference string: ");
    for (int i = 0; i < len; i++) scanf("%d", &ref[i]);
    for (int i = 0; i < n; i++) frame[i] = -1;
    int freq[10] = {0}, loaded[10] = {0}, clock = 0; /* freq: uses since loaded, loaded: load time (tie-break FIFO) */
    printf("\nPage | Frames\n");
    for (int i = 0; i < len; i++) {
        int hit = -1, fault = 0;
        clock++;
        for (int j = 0; j < n; j++) if (frame[j] == ref[i]) { hit = j; break; }
        if (hit >= 0) freq[hit]++;
        else {
            int victim = -1;
            for (int j = 0; j < n; j++) if (frame[j] == -1) { victim = j; break; }
            if (victim == -1) {
                victim = 0;
                for (int j = 1; j < n; j++)
                    if (freq[j] > freq[victim] || (freq[j] == freq[victim] && loaded[j] < loaded[victim])) victim = j;
            }
            frame[victim] = ref[i]; freq[victim] = 1; loaded[victim] = clock; faults++; fault = 1;
        }
        printf("%3d : ", ref[i]);
        for (int j = 0; j < n; j++) { if (frame[j] == -1) printf(" - "); else printf("%3d ", frame[j]); }
        printf("%s\n", fault ? " <- page fault" : "");
    }
    printf("\nTotal number of page faults = %d\n", faults);
    return 0;
}

```

Sample output (input shown as typed)

Enter number of frames: 3
Enter length of reference string: 16
Enter reference string: 2 5 2 8 5 4 1 2 3 2 6 1 2 5 9 8

Page		Frames	
2	:	2	- - <- page fault
5	:	2	5 - <- page fault
2	:	2	5 -
8	:	2	5 8 <- page fault
5	:	2	5 8
4	:	4	5 8 <- page fault
1	:	4	1 8 <- page fault
2	:	4	1 2 <- page fault
3	:	3	1 2 <- page fault
2	:	3	1 2
6	:	3	1 6 <- page fault
1	:	3	1 6
2	:	3	2 6 <- page fault
5	:	5	2 6 <- page fault
9	:	5	2 9 <- page fault
8	:	5	8 9 <- page fault

Total number of page faults = 12

Q3 - Oral/Viva: see viva questions at the end.

Q3 - Viva questions and answers

Deadlock / Banker's

Q. What is a deadlock? Four necessary conditions?

A. Processes wait forever for resources held by each other. Mutual exclusion, hold and wait, no preemption, circular wait.

Q. Need matrix formula?

A. $\text{Need} = \text{Max} - \text{Allocation}$.

Q. Safe state?

A. A state where a sequence of all processes exists so each can finish with available + released resources (safe sequence).

Q. Resource-Request algorithm steps?

A. Check $\text{Request} \leq \text{Need}$, then $\text{Request} \leq \text{Available}$; pretend to allocate; run safety algorithm; grant only if still safe.

Q. Avoidance vs prevention?

A. Avoidance (Banker's) uses advance info to stay in safe states; prevention breaks one of the four conditions.

CPU scheduling

Q. FCFS drawback?

A. Convoy effect - short jobs wait behind long ones.

Q. Why is SJF optimal? Its problem?

A. Gives minimum average waiting time; needs burst prediction and can starve long jobs.

Q. Preemptive SJF is called?

A. Shortest Remaining Time First (SRTF).

Q. Round Robin and time quantum?

A. Each process gets a fixed quantum in a circular queue. Very large quantum = FCFS; very small = too many context switches.

Q. Priority scheduling problem and fix?

A. Starvation; fixed by aging.

Q. Turnaround vs waiting time?

A. $\text{TAT} = \text{completion} - \text{arrival}$; $\text{WT} = \text{TAT} - \text{burst}$.

Disk scheduling

Q. Seek time?

A. Time to move the head to the target cylinder.

Q. SSTF drawback?

A. Starvation of far requests.

Q. SCAN vs C-SCAN vs LOOK?

A. SCAN sweeps to disk end then reverses; C-SCAN sweeps one way, jumps back, gives uniform wait; LOOK reverses at the last request instead of the disk end.

Page replacement / memory

Q. Page fault? Demand paging?

A. Access to a page not in memory; pages are loaded only when referenced.

Q. Belady's anomaly?

A. More frames can give more faults - occurs with FIFO.

Q. Optimal / LRU / MFU?

A. Optimal replaces the page not used for the longest future time (needs future knowledge); LRU replaces the least recently used; MFU replaces the most frequently used (assumes low-count pages are new).

File allocation

Q. Sequential / linked / indexed?

A. Sequential: contiguous blocks (fast access, external fragmentation). Linked: each block points to next (no fragmentation, no direct access). Indexed: index block lists all blocks (direct access, index overhead).

Q. Bit vector?

A. One bit per block (1 = allocated, 0 = free) to track free space.

Process system calls

Q. fork() return values?

A. 0 in child, child PID in parent, -1 on error.

Q. wait()?

A. Parent blocks until a child terminates; prevents zombies.

Q. Orphan vs zombie?

A. Orphan: parent exits first, child is adopted by init/systemd. Zombie: child exited but parent has not called wait().

Q. exec family / execve?

A. Replaces the current process image with a new program; same PID.

Q. nice()?

A. Changes process niceness; range -20 (highest priority) to 19 (lowest); lowering needs root.